



المدرسة العليا
للأساتذة - مراكش
ÉCOLE NORMALE
SUPÉRIEURE - MARRAKECH

Université Cadi Ayyad
École Normale Supérieure
Département d'Informatique

Licence d'éducation : Spécialité Enseignement Secondaire - Informatique
Semestre S6
Module : Développement Web et Mobile 2

Rapport de Projet de Fin de Module

Gestion de dépenses personnelles

Réalisé par

Hafssa CHKOUKED

Encadré par

Pr. Mohamed Lachgar

Année universitaire : 2025–2026

Déclaration

Je soussignée, **Hafssa CHKOUKED**, de l'École Normale Supérieure, Université Cadi Ayyad, Département d'Informatique, déclare que le présent rapport, y compris l'ensemble des textes, figures, tableaux, extraits de code et illustrations, constitue le fruit de mon travail personnel. Les sources externes utilisées ont fait l'objet de citations et références explicites. Je reconnais que tout manquement à cet engagement relèverait du plagiat, considéré comme une infraction académique passible de sanctions.

J'autorise la mise à disposition d'un exemplaire de ce rapport à des fins pédagogiques, au bénéfice des futurs étudiants et chercheurs, et j'accepte qu'il soit consulté dans l'intérêt général de l'Enseignement Supérieur et de la Recherche.

Hafssa CHKOUKED

14 mai 2026

(Signature)

Remerciements

Il m'est particulièrement agréable d'exprimer, à travers ces quelques lignes, ma profonde reconnaissance à mon encadrant pédagogique, **Pr.Mohamed Lachgar**, pour l'accompagnement de grande qualité dont j'ai bénéficié tout au long de la réalisation de ce projet intitulé « Gestion de dépenses personnelles ».

Je tiens à lui adresser mes sincères remerciements pour sa disponibilité constante, son écoute attentive ainsi que pour la pertinence de ses conseils, qui ont grandement contribué à orienter et enrichir ma réflexion. Son encadrement rigoureux, allié à sa bienveillance, m'a permis d'aborder ce travail avec méthode, confiance et exigence.

Ses remarques constructives, ses orientations éclairées et son soutien tout au long des différentes phases du projet ont été d'une aide précieuse pour surmonter les difficultés rencontrées et atteindre les objectifs fixés.

Je lui témoigne ainsi ma profonde gratitude et mon respect le plus sincère.

Abstract

This project, part of the *Web and Mobile Development 2* module, implements a multi-platform personal expense management system comprising an Android app (Java), a Node.js/Express REST API (SQLite), and a React web interface. Authentication uses JWT and bcrypt, respecting the four-table constraint. All endpoints have been tested with curl and Postman. The application successfully tracks expenses, budgets, and statistics with real-time notifications. This educational full-stack solution can be extended with geolocation, iOS support, or data export.

Keywords : expense management, React, Node.js, Android, REST API, JWT, SQLite

Résumé

Ce projet (module Développement Web et Mobile 2) fournit une application multi-plateforme de gestion de dépenses personnelles : Android (Java), API Node.js/Express, base SQLite (4 tables), interface web React. L'authentification repose sur JWT et bcrypt. Les tests (curl, Postman, émulateur) valident la conformité au cahier des charges. Perspectives : déploiement en ligne, version iOS, géolocalisation.

Mots-clés : gestion de dépenses, React, Node.js, Android, API REST, JWT, SQLite

Liste des abréviations

JWT	JSON Web Token
API	Application Programming Interface
REST	Representational State Transfer
CRUD	Create, Read, Update, Delete
UML	Unified Modeling Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
SQLite	Structured Query Language Lite

Table des figures

1.1	Cycle Scrum et ses principales étapes.	5
2.1	Diagramme de cas d'utilisation – Gestion de dépenses personnelles	9
4.1	Organisation en couches du système	16
4.2	Diagramme de séquence – Authentification	18
4.3	Diagramme de séquence – Ajout d'une dépense	19
4.4	Diagramme de séquence – Consultation du budget	20
4.5	Diagramme de classes – modèle de données	21
5.1	Écran de connexion (Android)	26
5.2	Écran de connexion (Web React)	27
5.3	Tableau de bord web (React)	27
5.4	Tableau de bord mobile (Android) – Vue des statistiques	28
5.5	Liste des dépenses (Web React)	28
5.6	Liste des dépenses avec filtre (Android)	29
5.7	Dialogue d'ajout de dépense (Android)	30
5.8	Gestion des catégories (Web React)	30
5.9	Gestion des catégories (Android)	31
5.10	Page de gestion du budget (Web React)	32
5.11	Suivi du budget (Android)	32

Liste des tableaux

1.1	Planification des sprints	6
3.1	Analyse comparative des solutions de gestion de dépenses	13
A.1	Routes de l'API REST (détailé)	45
A.2	Liste des codes HTTP et des messages d'erreur	47

Table des matières

Abstract	iii
Résumé	iv
Liste des abréviations	v
Table des figures	vi
Liste des tableaux	vii
Introduction Générale	1
1 Contexte général du projet	3
1.1 Introduction	3
1.2 Présentation du projet	3
1.2.1 Sujet du projet	3
1.2.2 Intérêt du projet	3
1.3 Problématique et état de l'existant	4
1.4 Objectifs du projet	4
1.4.1 Objectifs fonctionnels	4
1.4.2 Objectifs techniques	4
1.4.3 Contraintes	5
1.5 Démarche de gestion de projet	5
1.5.1 Méthodologie adoptée	5
1.5.2 Planification du projet	5
1.6 Conclusion	6
2 Étude préliminaire et fonctionnelle	7
2.1 Introduction	7
2.2 Étude des besoins	7
2.2.1 Besoins fonctionnels	7
2.2.2 Besoins non fonctionnels	8
2.3 Identification des acteurs	8
2.4 Cas d'utilisation et diagrammes	8
2.5 Conclusion	10
3 État de l'art	11
3.1 Introduction	11
3.2 Travaux connexes et solutions existantes	11
3.2.1 Applications mobiles grand public	11
3.2.2 Services web intégrés aux banques	12
3.2.3 Solutions open source	12

3.2.4	Technologies et frameworks	12
3.3	Analyse comparative	13
3.3.1	Grille d'évaluation des solutions	13
3.3.2	Tableau comparatif détaillé	13
3.4	Analyse des lacunes et opportunités	14
3.5	Conclusion	14
4	Analyse et conception	15
4.1	Introduction	15
4.2	Formalisme de modélisation (UML)	15
4.2.1	Motivation et choix	15
4.2.2	Diagrammes retenus	15
4.3	Architecture logicielle	16
4.3.1	Approche architecturale	16
4.3.2	Schéma de l'architecture et organisation en couches	16
4.3.3	Discussion	17
4.4	Diagrammes de conception	17
4.4.1	Diagrammes de séquence	17
4.4.2	Diagramme de classes	20
4.5	Conclusion	22
5	Technologies et réalisation	23
5.1	Introduction	23
5.2	Choix techniques et outils	23
5.2.1	Langages, Frameworks, Bases de données	23
5.2.2	Outils de développement (IDE, gestion de versions)	24
5.3	Architecture de déploiement	24
5.4	Implémentation et interfaces	25
5.4.1	Structure du code	25
5.4.2	Interfaces utilisateur (captures d'écran)	25
5.4.3	Fonctionnalités principales implémentées	32
5.5	Stratégie de tests	33
5.5.1	Tests unitaires	33
5.5.2	Tests d'intégration	33
5.5.3	Tests système	33
5.5.4	Outils de test et plan de validation	34
5.6	Conclusion	34
	Conclusion générale	35
	Bibliographie	38
	Appendices	39
A	Annexes (Facultatif)	39

Introduction Générale

Cette section introductive présente une vision globale du projet « Gestion de dépenses personnelles » et des travaux associés. Plusieurs éléments y sont abordés.

Contexte et champ d'application Le développement d'applications multi-plateformes est au cœur des préoccupations modernes en génie logiciel, notamment pour répondre à des besoins quotidiens comme le suivi des finances personnelles. Ce projet s'inscrit dans le cadre du module « Développement Web et Mobile 2 » de la Licence d'éducation – Spécialité Enseignement Secondaire – Informatique à l'École Normale Supérieure de Marrakech. L'objectif est de concevoir une solution complète composée d'une application mobile Android (Java), d'une API backend (Node.js avec SQLite) et d'une interface web (React), le tout respectant des contraintes pédagogiques strictes (quatre tables maximum, architecture claire, fonctionnalités réalistes).

Description du problème De nombreuses personnes ont du mal à maîtriser leurs dépenses et à suivre un budget mensuel de manière intuitive. Les solutions existantes sont souvent trop complexes, payantes, ou ne proposent pas à la fois une application mobile et un tableau de bord web. Le problème central est donc de fournir un outil simple, sécurisé et multi-plateforme permettant à un utilisateur d'enregistrer ses dépenses, de définir un budget, de visualiser des statistiques et d'être alerté en cas de dépassement.

Objectifs du projet Le projet vise à réaliser un système fonctionnel répondant au cahier des charges suivant :

- Une application mobile Android permettant l'authentification, l'ajout/modification/suppression de dépenses, des filtres (date, catégorie), la définition d'un budget mensuel et l'affichage de graphiques (répartition par catégorie, dépenses mensuelles).
- Une API REST (Node.js) assurant la persistance des données (SQLite), la validation serveur, l'authentification par JWT, la gestion des erreurs et une historisation minimale.
- Une interface web React offrant un tableau de bord avec statistiques (graphiques), la gestion des entités (CRUD), des filtres et un suivi des statuts.

Les contraintes non fonctionnelles (interface claire, sécurité, requêtes préparées, code structuré) doivent également être respectées.

Approche et méthodologie Pour mener à bien ce projet, une méthodologie agile Scrum a été adoptée. Le travail a été découpé en cinq sprints couvrant l'installation de l'environnement, le développement du backend, la réalisation de l'interface web, la création de l'application Android et enfin l'intégration/optimisation. Chaque sprint a donné lieu à des livrables intermédiaires et à des revues avec l'encadrant.

Résultats et interprétations attendus À l'issue du projet, nous disposons d'une application entièrement fonctionnelle où l'utilisateur peut se connecter, gérer ses dépenses et son budget, visualiser des graphiques dynamiques et recevoir des notifications en cas de dépassement. Les tests effectués (via 'curl', Postman, et les interfaces web/mobile) confirment la bonne intégration des trois composants et la conformité au cahier des charges. Le rapport documente l'ensemble du processus, des choix techniques aux captures d'écran des réalisations.

Organisation du rapport Le présent rapport est structuré comme suit :

- Le chapitre 1 dresse le contexte général du projet, la problématique, les objectifs et la démarche de gestion.
- Le chapitre 2 présente l'étude préliminaire et fonctionnelle (besoins, acteurs, cas d'utilisation, processus métier).
- Le chapitre 3 expose un état de l'art des solutions existantes et une analyse comparative.
- Le chapitre 4 détaille l'analyse et la conception (architecture, diagrammes UML, classes, séquences).
- Le chapitre 5 décrit les technologies utilisées, l'implémentation, les interfaces et la stratégie de tests.
- Enfin, une conclusion générale revient sur les apports du projet et propose des perspectives d'amélioration.

Chapitre 1

Contexte général du projet

1.1 Introduction

Ce chapitre a pour objectif de situer le projet « Gestion de dépenses personnelles » dans son contexte académique et technique. Il vise à présenter les motivations ayant conduit à sa réalisation ainsi que les besoins auxquels il répond. Nous y exposons également la problématique identifiée, les objectifs fonctionnels et techniques du projet, ainsi que les contraintes associées à sa mise en œuvre.

Par ailleurs, ce chapitre introduit la méthodologie de gestion de projet adoptée et la planification des différentes phases de développement. Il constitue ainsi une base essentielle pour la compréhension globale du projet et prépare le lecteur aux aspects fonctionnels et techniques détaillés dans les chapitres suivants.

1.2 Présentation du projet

1.2.1 Sujet du projet

Ce projet s'inscrit dans le cadre du module « Développement Web et Mobile 2 » de la Licence d'éducation – Spécialité Enseignement Secondaire – Informatique à l'École Normale Supérieure de Marrakech. Il a pour objectif la conception et la réalisation d'une application complète dédiée à la gestion des dépenses personnelles.

La solution proposée repose sur une architecture multi-plateforme composée de trois volets principaux :

- Une application **mobile Android**, développée en Java, permettant à l'utilisateur de saisir, modifier et consulter ses dépenses, de définir un budget mensuel et de visualiser des statistiques.
- Une **API backend** développée en Node.js avec Express, assurant la gestion des données, la logique métier et la sécurisation des échanges via une authentification basée sur les JSON Web Tokens (JWT).
- Une **interface web** développée en React, destinée à l'administration et à la visualisation des données sous forme de tableaux de bord et de graphiques.

Dans un souci de simplicité pédagogique, le projet respecte certaines contraintes, notamment la limitation à quatre tables dans la base de données, l'adoption d'une architecture claire et la mise en œuvre de fonctionnalités pertinentes et réalistes.

1.2.2 Intérêt du projet

La gestion des finances personnelles constitue un enjeu important dans la vie quotidienne. De nombreux utilisateurs rencontrent des difficultés à suivre leurs dépenses et à maîtriser leur budget de manière efficace.

Les solutions existantes, bien que nombreuses, présentent souvent certaines limites : complexité excessive, dépendance à des services externes, fonctionnalités non adaptées aux besoins réels ou encore coût d'utilisation. Dans ce contexte, le présent projet propose une alternative simple, accessible et adaptée.

Il offre une solution intégrée permettant :

- un suivi des dépenses en mobilité grâce à l'application Android,
- une visualisation détaillée des données via une interface web,
- une gestion personnalisée du budget avec alertes en cas de dépassement.

Sur le plan pédagogique, ce projet permet également de consolider les compétences en développement full-stack, en intégration de systèmes et en respect des bonnes pratiques de conception logicielle.

1.3 Problématique et état de l'existant

Avant de concevoir la solution proposée, une analyse des applications existantes de gestion de budget a été réalisée. Parmi les outils étudiés, on peut citer des applications telles que Linxo, Bankin' ou encore YNAB.

Cette étude a permis de mettre en évidence plusieurs limites :

- **Complexité fonctionnelle** : certaines applications proposent un grand nombre de fonctionnalités, rendant leur utilisation difficile pour un usage simple.
- **Dépendance aux services externes** : la synchronisation avec les comptes bancaires nécessite souvent l'utilisation d'API propriétaires.
- **Accessibilité limitée** : plusieurs solutions sont exclusivement mobiles et ne proposent pas d'interface web complète.
- **Coût** : certaines applications fonctionnent selon un modèle payant ou freemium.

Face à ces limites, il apparaît nécessaire de proposer une solution alternative, simple, gratuite et offrant un contrôle total des données. Le projet développé répond à ces besoins en proposant une application légère, indépendante et accessible sur plusieurs plateformes.

1.4 Objectifs du projet

Les objectifs du projet sont structurés en trois catégories principales :

1.4.1 Objectifs fonctionnels

- Mettre en place un système d'authentification sécurisé (inscription et connexion).
- Permettre la gestion des catégories de dépenses.
- Enregistrer et consulter les dépenses avec des filtres (date, catégorie).
- Définir et suivre un budget mensuel.
- Fournir des tableaux de bord avec des statistiques et des graphiques.
- Générer des alertes en cas de dépassement de budget.

1.4.2 Objectifs techniques

- Développer une API RESTful robuste avec Node.js et Express.
- Utiliser une base de données SQLite respectant la contrainte des quatre tables.

- Mettre en œuvre une authentification sécurisée basée sur JWT et le hachage des mots de passe.
- Concevoir une interface web dynamique avec React.
- Développer une application Android performante intégrant Retrofit, RecyclerView et des bibliothèques de visualisation.
- Assurer la sécurité et la fiabilité des données.

1.4.3 Contraintes

- **Temporelles** : réalisation du projet sur une durée limitée (un semestre).
- **Techniques** : maîtrise de plusieurs technologies (Android, Node.js, React).
- **Organisationnelles** : travail individuel avec un suivi régulier.

1.5 Démarche de gestion de projet

1.5.1 Méthodologie adoptée

Afin de mener à bien ce projet, une méthodologie agile de type **Scrum** a été adoptée. Cette approche favorise le développement itératif, l'adaptabilité aux changements et la livraison progressive des fonctionnalités.

Les rôles principaux sont définis comme suit :

- **Product Owner** : l'encadrant pédagogique, responsable de la validation des fonctionnalités.
- **Scrum Master** : l'étudiant, chargé de l'organisation et du suivi du projet.
- **Équipe de développement** : l'étudiant lui-même, intervenant sur l'ensemble des composants.

Chaque itération (sprint) permet de développer un ensemble cohérent de fonctionnalités et de procéder à leur validation.

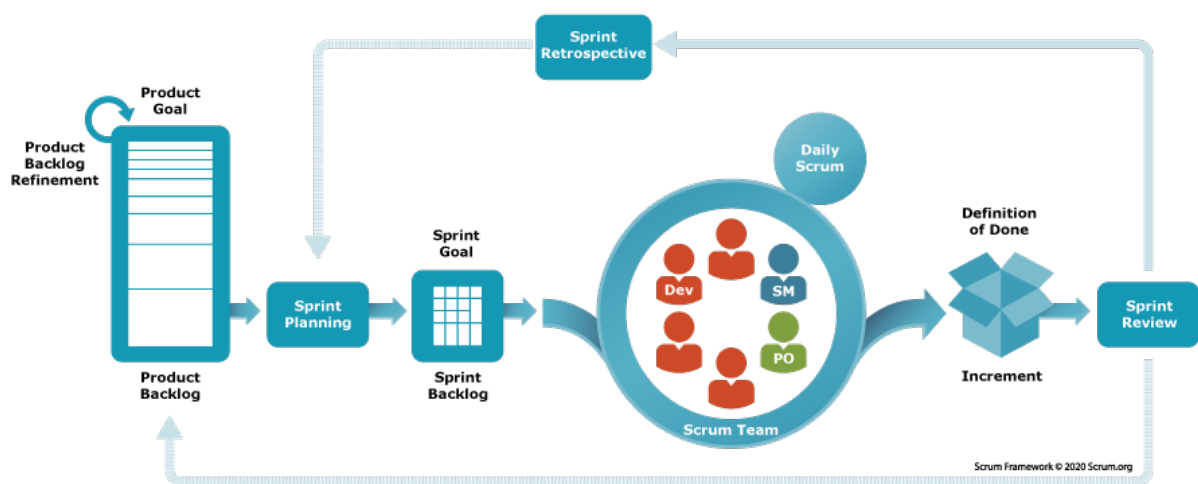


Figure 1.1 : Cycle Scrum et ses principales étapes.

1.5.2 Planification du projet

Le projet a été structuré en cinq sprints, chacun ayant des objectifs précis et des livrables associés.

Table 1.1 : Planification des sprints

Sprint	Objectifs	Durée	Livrables
1	Mise en place de l'environnement de développement et configuration des outils.	2 semaines	Environnement prêt, dépôt initial
2	Développement du backend et des fonctionnalités principales.	3 semaines	API fonctionnelle
3	Conception de l'interface web et intégration des graphiques.	2 semaines	Application web opérationnelle
4	Développement de l'application Android.	3 semaines	Application mobile fonctionnelle
5	Finalisation, tests et rédaction du rapport.	2 semaines	Version finale complète

1.6 Conclusion

Ce chapitre a permis de présenter le cadre général du projet, en mettant en évidence son contexte, sa problématique, ses objectifs ainsi que les contraintes associées. La méthodologie adoptée et la planification définie constituent une base solide pour la réalisation du projet.

Dans le chapitre suivant, nous aborderons l'étude fonctionnelle détaillée, en identifiant les besoins des utilisateurs, les cas d'utilisation ainsi que les processus métier nécessaires à la conception du système.

Chapitre 2

Étude préliminaire et fonctionnelle

2.1 Introduction

Ce chapitre a pour objectif de **cadre les besoins** auxquels le projet «Gestion de dépenses personnelles» doit répondre, qu'ils soient liés aux fonctionnalités principales, aux contraintes techniques ou à l'expérience utilisateur. Il permet également de clarifier l'identité des différents acteurs, ainsi que leur rôle dans le système. Enfin, il détaille la logique de fonctionnement au travers de cas d'utilisation et de processus métier.

2.2 Étude des besoins

2.2.1 Besoins fonctionnels

Les besoins fonctionnels décrivent ce que le système doit faire. Pour notre application, on distingue les fonctionnalités suivantes :

- **Gestion des comptes utilisateurs** : inscription, connexion sécurisée (JWT), déconnexion.
- **Gestion des catégories de dépenses** : ajout, modification, suppression de catégories (avec code couleur pour l'interface mobile).
- **Gestion des dépenses** : ajout d'une dépense (montant, date, catégorie, note facultative), modification, suppression, consultation de la liste.
- **Filtrage des dépenses** : par intervalle de dates, par catégorie (sur mobile et sur web).
- **Budget mensuel** : définition d'un budget par mois, visualisation du montant consommé et du reste, alerte en cas de dépassement.
- **Statistiques** : répartition des dépenses par catégorie (camembert) et évolution mensuelle (histogramme), sur mobile et sur web.
- **Notifications** : alerte locale sur l'appareil mobile lorsque le budget est dépassé.

Scénario d'utilisation concret (ajout d'une dépense) :

1. L'utilisateur se connecte avec son adresse e-mail et son mot de passe.
2. Il accède à l'écran «Dépenses» sur son mobile ou à la page correspondante sur le web.
3. Il clique sur le bouton «Ajouter une dépense».
4. Il saisit le montant, la date (par défaut la date courante), choisit une catégorie dans une liste déroulante, et ajoute éventuellement une note.
5. Il valide. La dépense est enregistrée en base via l'API et apparaît immédiatement dans la liste.
6. Le tableau de bord (web et mobile) se met à jour : les graphiques sont recalculés et le suivi du budget est actualisé.

2.2.2 Besoins non fonctionnels

Les besoins non fonctionnels concernent la qualité, la sécurité et les performances du système.

- **Sécurité** : les mots de passe sont hachés (bcrypt) avant d'être stockés. L'authentification utilise des jetons JWT (expiration 24h). Les requêtes API sont protégées par ce jeton, sauf les routes d'inscription et de connexion.
- **Intégrité des données** : la base SQLite impose des contraintes d'unicité et de clés étrangères. Les requêtes SQL sont préparées (via db.run / db.all) pour éviter les injections.
- **Simplicité pédagogique** : la base de données ne comporte que quatre tables (users, expense_categories, expenses, monthly_budgets); l'architecture est clairement séparée en couches (contrôleurs, routes, middleware).
- **Interface utilisateur** : l'application mobile suit les principes de Material Design (RecyclerView, FloatingActionButton). L'interface web est responsive, avec des graphiques Recharts.
- **Temps de réponse** : les appels API sont asynchrones (Retrofit côté Android, Axios côté web). Les listes sont paginées virtuellement (pas de limite fixe dans la version actuelle, mais l'utilisation de SQLite avec index assure des temps acceptables).
- **Performances** : le serveur Node.js traite les requêtes REST rapidement; la base SQLite est légère et adaptée à des volumes raisonnables de données.

2.3 Identification des acteurs

Le système comporte deux types d'acteurs :

- **Utilisateur (rôle par défaut)** :
 - Peut s'inscrire, se connecter, se déconnecter.
 - Gère ses propres catégories, dépenses et budgets.
 - Consulte ses propres statistiques (graphiques).
 - Reçoit des notifications sur mobile en cas de dépassement.
- **Administrateur (optionnel – mentionné dans le cahier des charges mais non implémenté faute de temps)** :
 - Aurait pu visualiser l'ensemble des utilisateurs et leurs données, gérer les rôles, etc.

Dans la version réalisée, seul l'utilisateur standard est présent. Chaque utilisateur ne peut accéder qu'à ses propres données (les requêtes SQL incluent toujours un filtre sur user_id).

2.4 Cas d'utilisation et diagrammes

Le diagramme de cas d'utilisation de la figure 2.1 présente les interactions possibles entre l'utilisateur et le système.

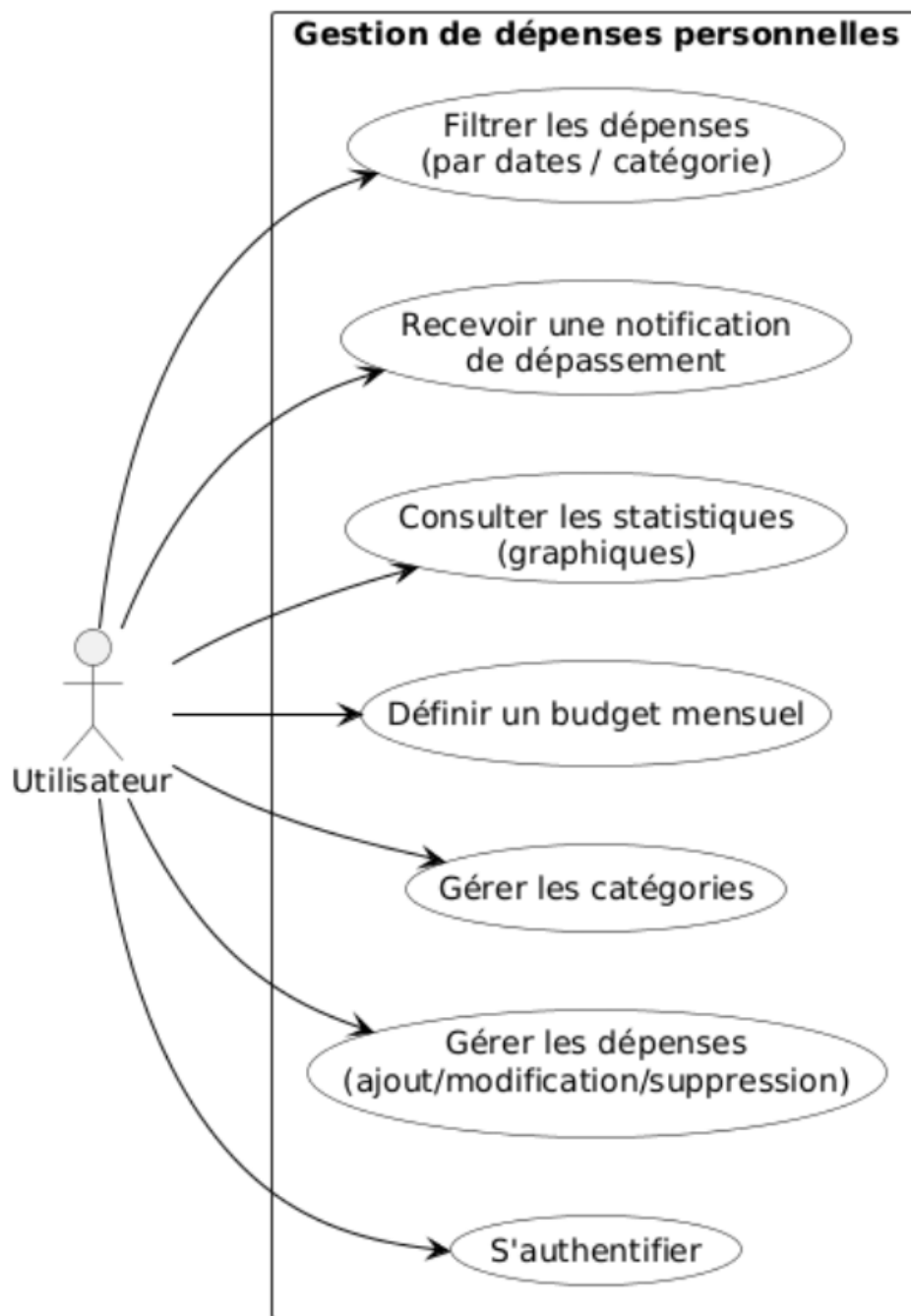


Figure 2.1 : Diagramme de cas d'utilisation – Gestion de dépenses personnelles

Liste textuelle des cas d'utilisation :

- **S'authentifier** : précondition – l'utilisateur n'est pas connecté. Il fournit son email et mot de passe. Le système vérifie les identifiants et retourne un jeton JWT.
- **Gérer les catégories** : l'utilisateur peut ajouter, modifier ou supprimer une catégorie. La suppression est interdite si des dépenses y sont associées.
- **Ajouter une dépense** : l'utilisateur saisit le montant, la date, la catégorie et une note. Le montant doit être >0 . La date doit être au format ISO.
- **Consulter les dépenses** : affichage d'une liste filtrée (par dates et/ou catégorie). L'utilisateur peut modifier ou supprimer chaque dépense.

- Définir un budget mensuel : l'utilisateur choisit un mois (AAAA-MM) et un montant. Un budget écrasé remplace l'ancien.
- Visualiser les statistiques : affichage d'un graphique en barres (dépenses mensuelles de l'année) et d'un camembert (répartition du mois courant).
- Recevoir une notification (système) : si la somme des dépenses du mois dépasse le budget défini pour ce mois, une notification locale est déclenchée sur le mobile.

Description du flux d'informations :

1. L'utilisateur clique sur "Ajouter dépense" (bouton flottant sur mobile, ou bouton sur web).
2. Un formulaire s'affiche (montant, date, catégorie, note).
3. Il saisit les données et valide.
4. Le client (Android ou React) envoie une requête POST à l'API (/api/expenses) avec le jeton JWT dans l'en-tête.
5. Le serveur vérifie la validité du jeton et que le montant est >0 .
6. Une insertion est effectuée dans la table `expenses`.
7. En cas de succès, le serveur retourne le code 201 (Created) avec l'identifiant de la nouvelle dépense.
8. Le client recharge la liste des dépenses et rafraîchit le tableau de bord (le fragment Budget et le fragment Statistiques se mettent à jour automatiquement).

Ce processus est identique quelle que soit la plateforme (Android, web) grâce à l'utilisation de la même API REST.

2.5 Conclusion

Ce chapitre a permis d'établir le cadre fonctionnel du projet. Les besoins fonctionnels (gestion des comptes, des catégories, des dépenses, du budget, des statistiques) et non fonctionnels (sécurité, simplicité, performance) ont été clairement définis. L'identification des acteurs se limite à l'utilisateur authentifié, seul rôle réellement mis en œuvre. Les cas d'utilisation et le diagramme d'activités de l'ajout d'une dépense illustrent la logique métier. Ces éléments serviront de base à l'état de l'art (chapitre 3), où nous confronterons notre approche aux solutions existantes, puis à la conception technique (chapitre 4).

Chapitre 3

État de l'art

3.1 Introduction

Le présent chapitre vise à positionner le projet «Gestion de dépenses personnelles» dans son contexte scientifique et technique, en s'appuyant sur un examen des approches et des travaux déjà réalisés dans le domaine de la gestion budgétaire personnelle. Il a pour rôle de situer notre démarche par rapport aux solutions existantes (applications mobiles, services web, outils de suivi financier) et d'aider à justifier la pertinence de notre proposition. Les éléments abordés comprennent une recherche bibliographique sur les applications de gestion de budget, une comparaison d'outils disponibles (propriétaires ou open source) et une mise en exergue des besoins encore non satisfaits, particulièrement dans le cadre d'un projet pédagogique visant à maîtriser le développement multi-plateforme.

3.2 Travaux connexes et solutions existantes

3.2.1 Applications mobiles grand public

De nombreuses applications tentent d'aider les particuliers à gérer leurs dépenses. Nous distinguons trois catégories : les applications mobiles grand public, les services web intégrés aux banques, et les solutions open source.

YNAB (You Need A Budget)

YNAB est basé sur la méthode des enveloppes : chaque euro est affecté à une catégorie. Elle propose une synchronisation bancaire via des APIs tierces (Plaid, etc.), une interface web et mobile, des objectifs d'épargne et des rapports détaillés. **Points forts** : méthodologie reconnue, automatisation, multi-plateforme. **Limites** : abonnement annuel (environ 100\$), complexité pour un usage simple, dépendance à des services externes pour la synchronisation.

Mint (Intuit)

Mint agrège automatiquement les transactions bancaires (États-Unis), catégorise les dépenses, envoie des alertes et propose un suivi de budget. **Points forts** : gratuit (financé par publicité et offres de produits financiers), interface intuitive. **Limites** : indisponible hors des États-Unis, modèle publicitaire intrusif, absence de contrôle sur l'utilisation des données personnelles.

Bankin' (leader européen)

Bankin' est une application française qui se connecte à plus de 400 banques en Europe. Elle analyse les dépenses, permet de définir des budgets, visualise des graphiques. **Points forts** : connectivité étendue, ergonomie, version gratuite. **Limites** : application mobile uniquement (pas de tableau de

bord web complet), nécessite des identifiants bancaires (risque de sécurité perçu par les utilisateurs), dépendance aux APIs bancaires.

3.2.2 Services web intégrés aux banques

De nombreuses banques (Crédit Agricole, BNP Paribas, Société Générale, etc.) proposent des fonctionnalités de catégorisation automatique dans leur espace client. **Points forts** : intégration native, gratuité, données fiables. **Limites** : outils fermés, non extensibles, ne permettent pas de saisir des transactions en espèces ou hors banque (ex. achats sur des places de marché). De plus, les exportations sont souvent limitées au format CSV sans API publique.

3.2.3 Solutions open source

Quelques projets open source permettent un contrôle total des données.

Firefly III

Firefly III est une application web auto-hébergée écrite en PHP (Laravel). Elle gère les comptes, les budgets, les factures, les transactions récurrentes, et offre une API REST complète. Des clients mobiles tiers existent mais ne couvrent pas toutes les fonctionnalités. **Points forts** : riche fonctionnellement, bonne documentation, communauté active. **Limites** : lourd à installer (nécessite PHP, MySQL/PostgreSQL), pas d'application mobile officielle, modèle de données complexe (plus de 15 tables).

GNUcash

GNUcash est un logiciel de comptabilité en partie double, disponible sur ordinateur (Windows, Linux, Mac). Il gère les comptes bancaires, les actions, les budgets. **Points forts** : puissant, standard comptable, open source. **Limites** : interface datée, courbe d'apprentissage élevée, absence d'application mobile ou d'API web.

Actual Budget

Actual Budget est une alternative open source à YNAB, basée sur Node.js et React, avec synchronisation bancaire (SimpleFin) et interface web. **Points forts** : auto-hébergement, design moderne. **Limites** : nécessite Docker ou Node.js avancé, pas d'application mobile native (seulement une PWA), communauté encore petite.

3.2.4 Technologies et frameworks

Une revue des technologies employées dans des projets similaires montre plusieurs tendances :

- **Backend** : PHP (Laravel, Symfony) – mature, mais souvent surdimensionné. Node.js (Express) – léger, événementiel, idéal pour des API REST. Java (Spring Boot) – robuste mais plus lourd.
- **Base de données** : MySQL/PostgreSQL – pour les projets avec plusieurs utilisateurs. SQLite – parfait pour des applications personnelles ou de petite taille, car zéro configuration.
- **Frontend web** : React – composants réutilisables, écosystème riche. Vue – plus simple, mais moins intégré avec Recharts. Angular – trop complexe pour un projet pédagogique.
- **Mobile** : Android (Java/Kotlin) avec Retrofit pour les appels API, MPAndroidChart pour les graphiques. Flutter – intéressant pour le cross-plateforme, mais nécessite l'apprentissage d'un nouveau langage (Dart).

Notre projet s'inscrit dans cette veine en choisissant une stack moderne mais minimaliste : Node.js/Express, SQLite, React, Android Java. Ce choix garantit une légèreté, une facilité de déploiement et une adéquation avec les objectifs pédagogiques.

3.3 Analyse comparative

3.3.1 Grille d'évaluation des solutions

Pour comparer objectivement les solutions, nous définissons les critères suivants :

1. **Contrôle des données** : l'utilisateur possède-t-il ses données (hébergement local) ou sont-elles stockées sur des serveurs tiers ?
2. **Interface mobile** : présence d'une application native (Android/iOS) ou seulement d'un site responsive ?
3. **Interface web** : tableau de bord accessible par navigateur pour analyse et reporting ?
4. **API REST** : possibilité d'interagir programmatiquement avec le système ?
5. **Simplicité (nombre de tables)** : pour un projet pédagogique, une faible complexité est recherchée.
6. **Notification en cas de dépassement** : fonctionnalité d'alerte.
7. **Coût** : gratuit, freemium ou payant.
8. **Adaptabilité pédagogique** : capacité à être utilisé comme support de cours pour le développement multi-plateforme.

3.3.2 Tableau comparatif détaillé

Le tableau 3.1 synthétise l'évaluation des principales solutions et de notre projet.

Table 3.1 : Analyse comparative des solutions de gestion de dépenses

Critère	YNAB	Bankin'	Firefly III	Actual Budget	Notre projet
Contrôle des données	Non (cloud)	Non (cloud)	Oui (auto-hébergement)	Oui (auto-hébergement)	Oui (SQlite mobile)
Application mobile	iOS/Android	iOS/Android	Non (PWA tierce)	PWA	Android (Java)
Interface web	Oui	Non	Oui	Oui	Oui (React)
API REST	Partielle	Partielle	Complète	Oui	Complète (Node.js)
Simplicité (nb tables)	Élevé (>20)	N/A	>15	>10	4
Notification dépassement	Oui	Oui	Oui	Oui	Oui (Android)
Coût	Payant (abonnement)	Freemium	Gratuit	Gratuit	Gratuit
Adaptabilité pédagogique	Faible	Faible	Moyenne	Moyenne	Élevée

Analyse : Aucune solution existante ne remplit simultanément tous les critères pour un projet pédagogique multi-plateforme. YNAB et Bankin' sont des produits finis non libres et complexes. Firefly III et Actual Budget sont open source mais leurs architectures sont trop lourdes pour une maîtrise rapide. Notre projet se distingue par une combinaison unique : contrôle total des données, double interface (mobile native et web) avec API REST légère, gratuité, et une complexité maîtrisée (4 tables). Il répond explicitement aux besoins d'un module universitaire où l'étudiant doit apprendre à intégrer trois composants.

3.4 Analyse des lacunes et opportunités

L'étude des travaux connexes met en évidence plusieurs lacunes que notre projet comble :

- **Manque de solutions pédagogiques intégrées** : La plupart des applications sont destinées à un usage final, non à l'apprentissage du développement full-stack.
- **Dépendance à des services externes** : Les outils modernes s'appuient sur des agrégateurs bancaires (Plaid, Finicity) qui ne sont pas gratuits ni disponibles dans tous les pays.
- **Absence d'API REST légère avec front mobile** : Firefly III a une API mais pas de client Android officiel ; les projets comme Actual Budget se concentrent sur le web.
- **Complexité des modèles de données** : Les applications professionnelles utilisent des dizaines de tables, ce qui rend l'appropriation difficile pour un étudiant.

Notre projet capitalise sur ces opportunités pour proposer une solution simple, extensible, et parfaitement adaptée à l'enseignement du développement web et mobile. Sa conception modulaire (backend API, frontend React, client Android) suit les standards de l'industrie tout en restant accessible.

3.5 Conclusion

Ce chapitre a dressé un panorama des principales applications de gestion de dépenses. Nous avons constaté que les outils propriétaires verrouillent les données et imposent des modèles économiques, tandis que les solutions open source sont souvent lourdes et dépourvues d'une application mobile moderne. Notre projet se distingue par :

- Une architecture totalement libre et auto-hébergée (SQLite, aucune dépendance externe).
- Une double interface (Android et React) couvrant les besoins des utilisateurs nomades et ceux qui préfèrent un tableau de bord sur écran large.
- La simplicité pédagogique : 4 tables, code structuré, authentification JWT, respect des bonnes pratiques.
- Une réponse aux besoins non satisfaits : légèreté, extensibilité, adaptabilité à un projet universitaire.

Cette analyse nous conforte dans nos choix technologiques (Node.js, SQLite, React, Android) qui seront détaillés dans le chapitre de conception (chapitre 4). La comparaison montre que notre proposition apporte une valeur ajoutée en termes de contrôle, de simplicité et d'ouverture. Nous allons maintenant passer à la modélisation et à la conception détaillée de la solution.

Chapitre 4

Analyse et conception

4.1 Introduction

Ce chapitre présente la traduction des besoins fonctionnels et non fonctionnels du projet «Gestion de dépenses personnelles» en une solution technique détaillée. Après avoir défini le formalisme de modélisation (UML), nous décrivons l'architecture logicielle retenue (monolithique en couches) ainsi que les diagrammes de conception (classes, séquences, déploiement). L'objectif est de fournir une base solide pour l'implémentation, en justifiant chaque choix à la lumière des contraintes pédagogiques (4 tables maximum, simplicité) et techniques (sécurité, performance, interopérabilité).

4.2 Formalisme de modélisation (UML)

4.2.1 Motivation et choix

Le langage UML (Unified Modeling Language) a été sélectionné pour sa capacité à représenter de manière normalisée à la fois la structure statique et le comportement dynamique du système. Il facilite la communication entre les différents acteurs (développeurs, encadrant) et garantit une documentation cohérente. Dans ce projet, UML permet de :

- Décrire les entités persistantes et leurs relations via un diagramme de classes.
- Illustrer les échanges entre les composants (client Android, interface web, API, base de données) au moyen de diagrammes de séquence.
- Visualiser les flux d'exécution pour les scénarios critiques (ajout d'une dépense, définition du budget).

4.2.2 Diagrammes retenus

Les diagrammes suivants ont été élaborés :

- **Diagrammes de séquence** : pour l'authentification, l'ajout d'une dépense, la consultation des statistiques et la gestion du budget.
- **Diagramme de classes** : modélisation des quatre tables de la base de données et de leurs relations.
- **Diagramme de déploiement** : pour décrire l'infrastructure matérielle et les processus.
- **Diagramme d'activités** (optionnel) : pour le processus d'ajout de dépense (similaire à celui du chapitre 2).

4.3 Architecture logicielle

4.3.1 Approche architecturale

Nous avons opté pour une **architecture monolithique en couches**. Ce choix est justifié par :

- **Simplicité** : le projet ne nécessite pas la complexité d'une architecture microservices (gestion de la découverte, du routage, des transactions distribuées).
- **Cohérence des données** : une seule base SQLite, facile à déployer et à sauvegarder.
- **Contraintes pédagogiques** : la taille réduite du projet (4 tables) rend une approche monolithique parfaitement adaptée.
- **Facilité de test** : l'ensemble des composants (API, web, mobile) communiquent via le même serveur, ce qui simplifie les tests d'intégration.

L'inconvénient d'une éventuelle difficulté à faire évoluer le système vers une architecture distribuée n'est pas rédhibitoire dans notre cadre académique.

4.3.2 Schéma de l'architecture et organisation en couches

L'architecture est divisée en trois grandes couches (voir figure 4.1) :

1. **Couche de présentation** :
 - Client Android (Java) – fournit une interface mobile.
 - Client web (React) – fournit une interface de bureau.
2. **Couche de logique métier (API)** :
 - Serveur Node.js avec Express.
 - Contrôleurs (auth, expenses, categories, budgets, stats).
 - Middleware d'authentification JWT, validation des données.
3. **Couche de données** :
 - Base de données SQLite (fichier `expenses.db`).
 - Modèles implicites via les requêtes SQL préparées.

Les deux clients communiquent exclusivement avec l'API via des requêtes HTTP (REST). L'API accède à la base de données via le module `sqlite3`. Le flux est unidirectionnel : présentation → métier → données.

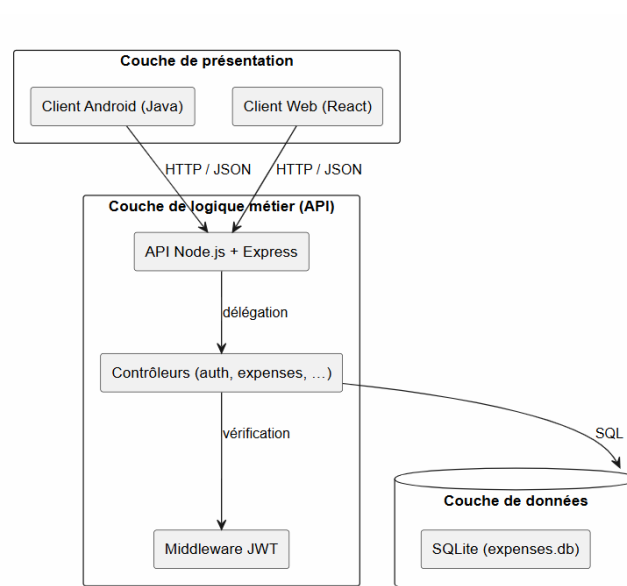


Figure 4.1 : Organisation en couches du système

4.3.3 Discussion

Cette architecture répond aux besoins non fonctionnels :

- **Sécurité** : le middleware JWT protège toutes les routes sensibles ; les mots de passe sont hachés (bcrypt).
- **Maintenabilité** : la séparation des préoccupations (contrôleurs, routes, modèles) facilite l'évolution.
- **Performance** : Node.js assure un temps de réponse faible ; SQLite est suffisant pour des centaines de requêtes simultanées.
- **Extensibilité** : l'ajout d'un nouveau client (par exemple iOS) nécessite seulement d'implémenter les appels à la même API.
- **Testabilité** : l'API peut être testée indépendamment avec curl ou Postman ; les clients utilisent des mocks.

4.4 Diagrammes de conception

4.4.1 Diagrammes de séquence

Authentification

La figure 4.2 montre l'enchaînement des interactions pour la connexion d'un utilisateur :

1. L'utilisateur saisit email/mot de passe dans le client (Android ou web).
2. Le client envoie une requête POST à `/api/auth/login`.
3. Le contrôleur `authController.login()` interroge la base via `db.get`.
4. Si le compte existe et le mot de passe correspond (`bcrypt.compareSync`), un jeton JWT est généré.
5. Le jeton est retourné au client, qui le stocke localement (`SharedPreferences` ou `localStorage`).

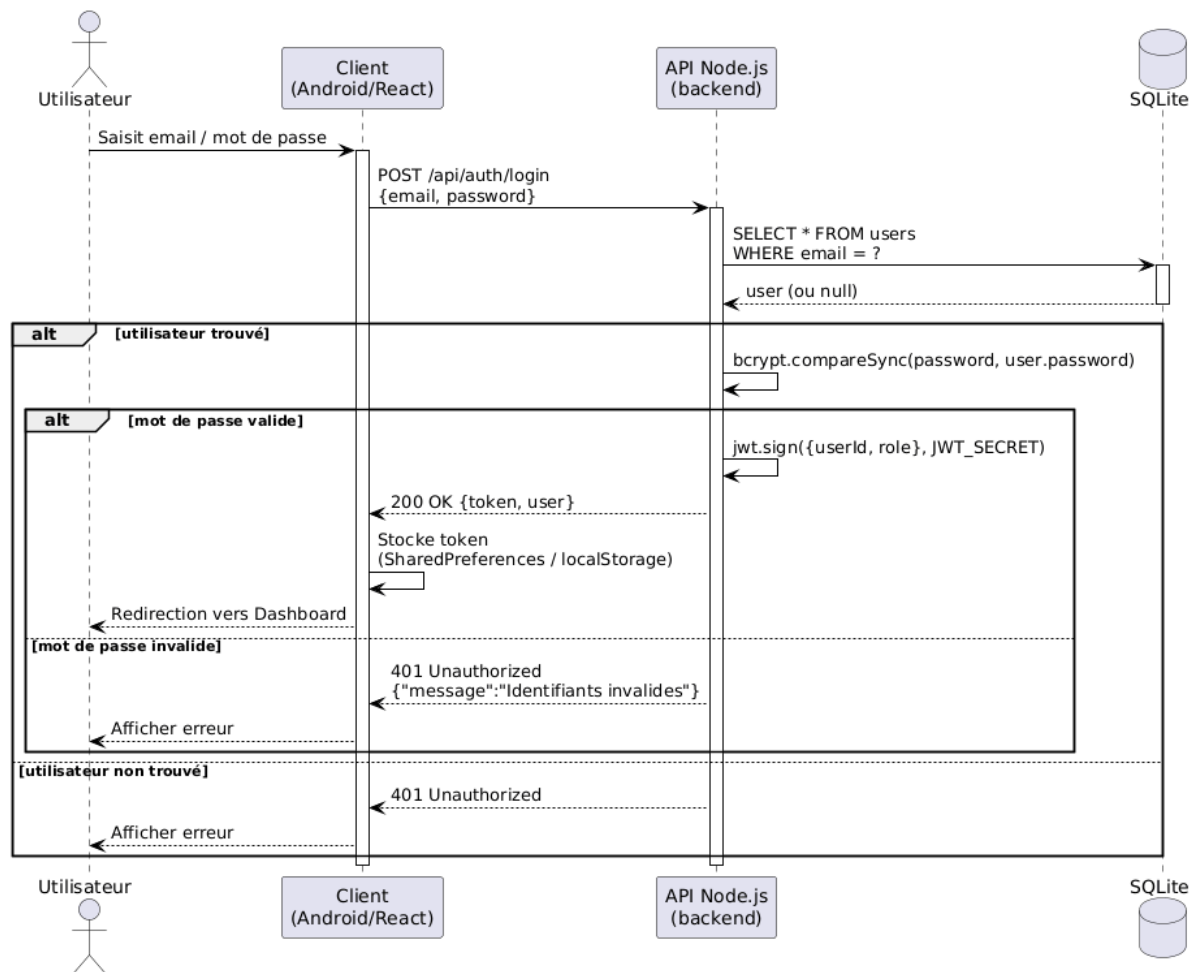


Figure 4.2 : Diagramme de séquence – Authentification

Ajout d'une dépense

Le scénario (figure 4.3) implique le client, l'API et la base :

1. L'utilisateur remplit le formulaire (montant, date, catégorie, note) et valide.
2. Le client envoie une requête POST à `/api/expenses` avec le jeton dans l'en-tête `Authorization`.
3. Le middleware `auth.js` vérifie le jeton et ajoute `req.userId`.
4. Le contrôleur `expenseController.createExpense()` valide le montant (>0) et exécute une insertion SQL.
5. La base retourne l'identifiant de la nouvelle dépense.
6. L'API répond avec un statut 201 (Created). Le client recharge la liste des dépenses.

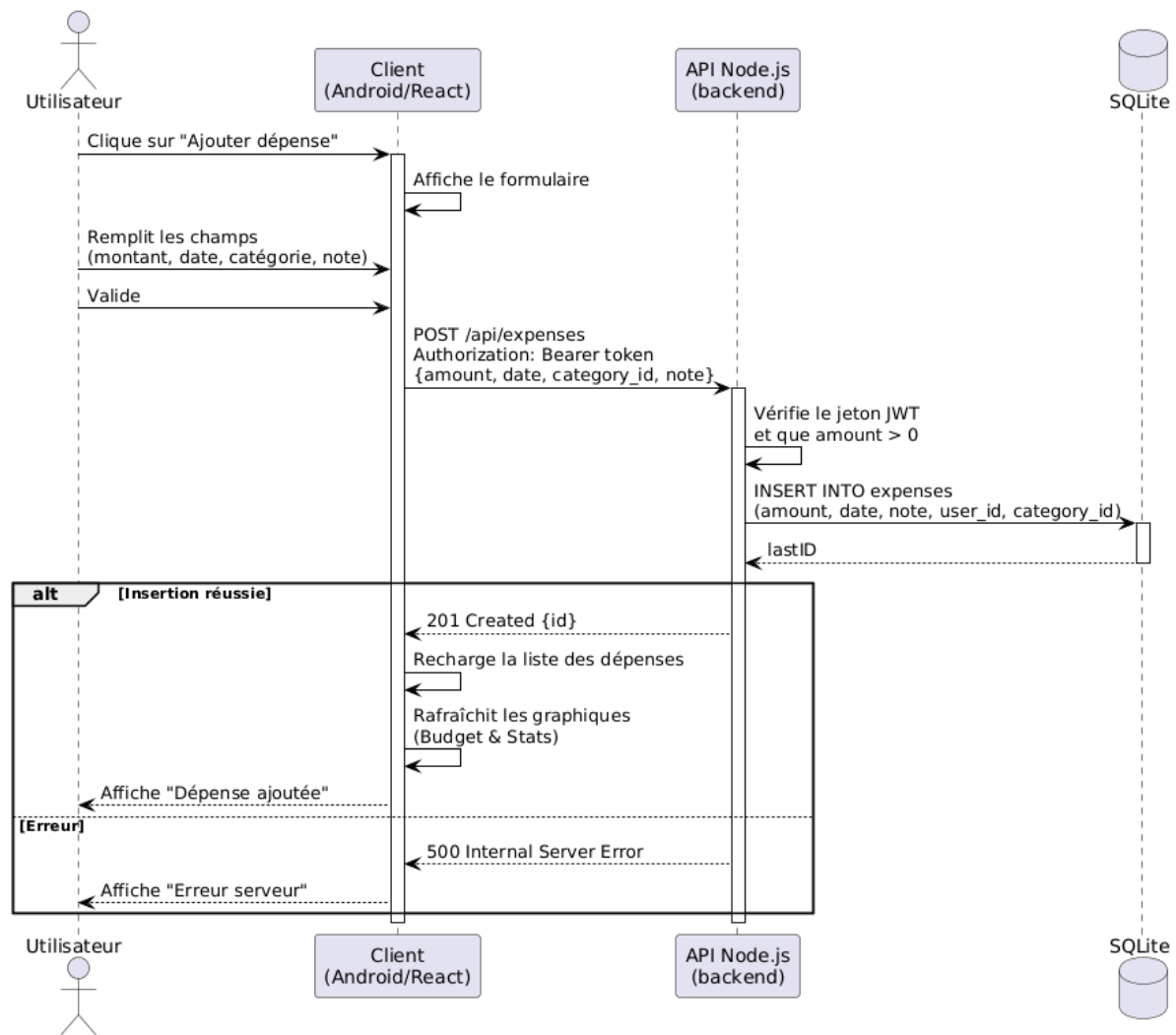


Figure 4.3 : Diagramme de séquence – Ajout d'une dépense

Définition et suivi du budget

La figure 4.4 illustre la double consultation (budget existant + dépenses du mois) pour mettre à jour l'interface.

1. Le fragment Budget appelle deux endpoints en parallèle : GET /budgets/YYYY-MM et GET /budgets/current-spending.
2. Le contrôleur budgetController renvoie respectivement le montant planifié et la somme des dépenses.
3. Le client calcule le reste, met à jour la barre de progression et déclenche une notification si dépassement.

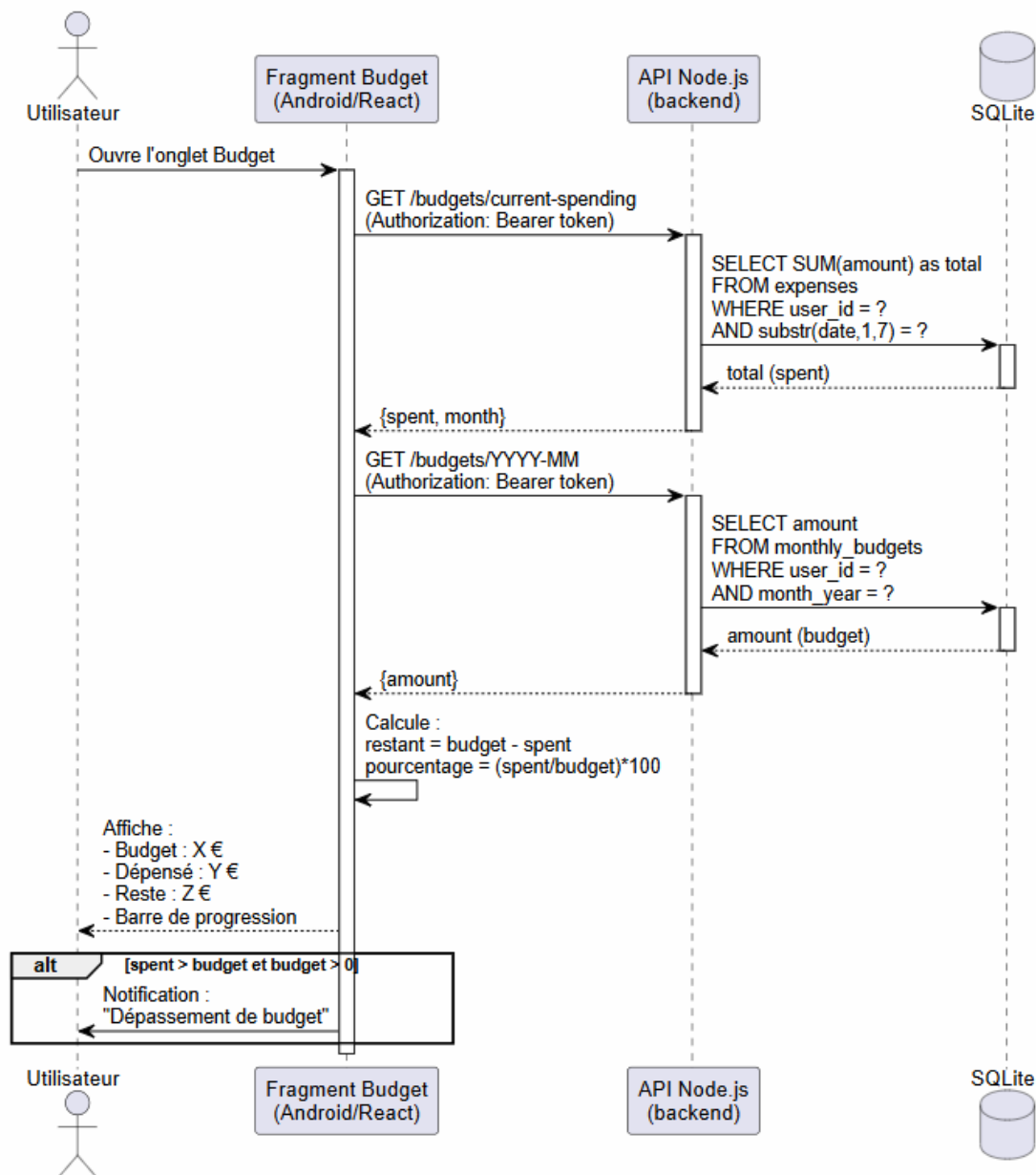


Figure 4.4 : Diagramme de séquence – Consultation du budget

4.4.2 Diagramme de classes

La figure 4.5 présente le diagramme de classes correspondant au modèle de données. Quatre classes principales persistent en base de données :

- **User** : attributs (id, email, password, name, role). Méthodes d'authentification (délégées au contrôleur).
- **Category** : attributs (id, name, color, user_id). Relation 1..* avec User (une catégorie appartient à un utilisateur). Une catégorie peut être associée à plusieurs Expense.
- **Expense** : attributs (id, amount, date, note, user_id, category_id). Relation * vers Category et User.
- **Budget** : attributs (id, month_year, amount, user_id). Relation * vers User.

Les cardinalités sont :

- Un User peut avoir 0..* Category (chaque utilisateur gère ses propres catégories).
- Un User peut avoir 0..* Expense.
- Un User peut avoir 0..* Budget (un seul par mois, mais la contrainte d'unicité est gérée en base).
- Une Expense appartient à exactement une Category et un User.

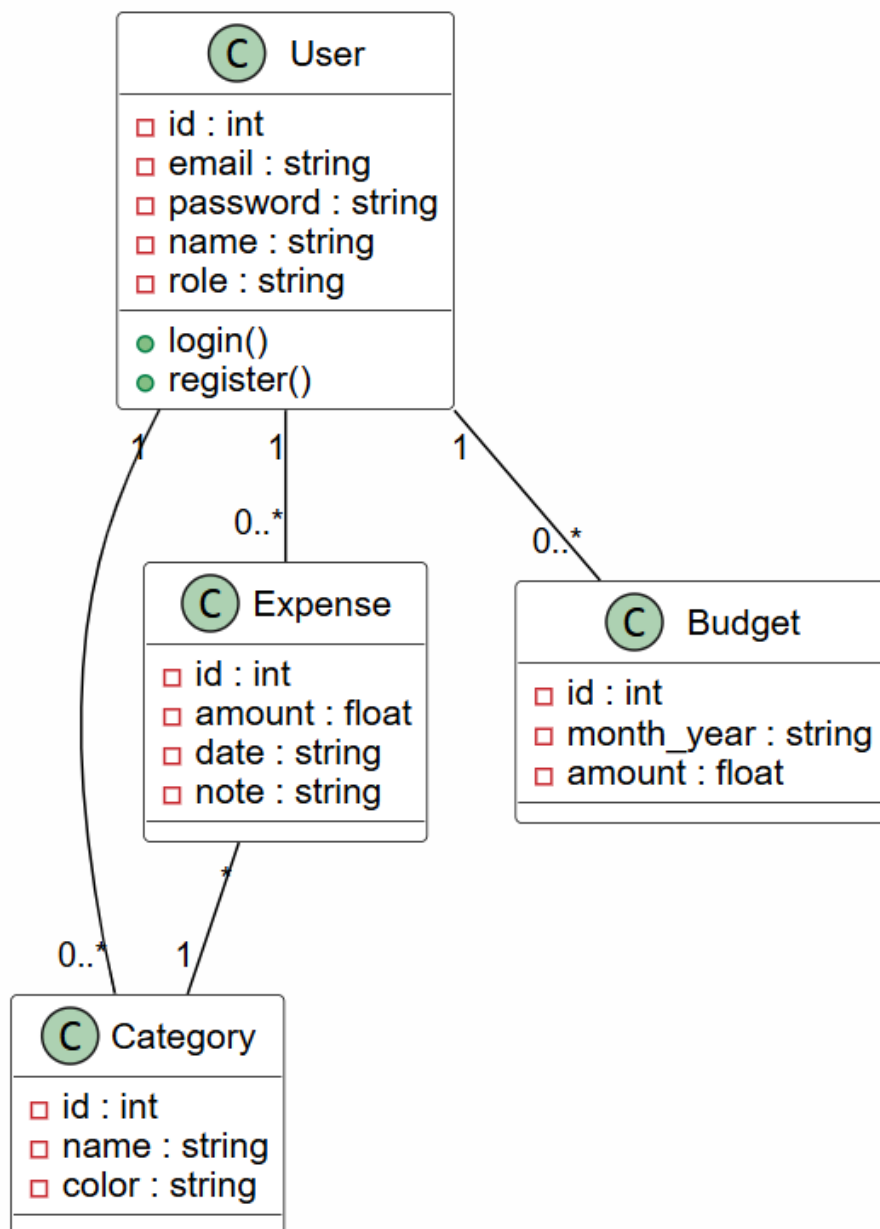


Figure 4.5 : Diagramme de classes – modèle de données

4.5 Conclusion

Ce chapitre a détaillé l'analyse et la conception du projet. Nous avons justifié le choix d'UML comme formalisme de modélisation, présenté une architecture monolithique en trois couches, et fourni les diagrammes de classes et de séquence pour les scénarios clés. Le diagramme de déploiement complète la vision technique. Ces modèles répondent aux besoins fonctionnels (authentification, gestion des dépenses, budget, statistiques) et non fonctionnels (sécurité, maintenabilité, simplicité). Ils constituent le socle sur lequel l'implémentation (chapitre 5) a été réalisée. Les choix effectués – notamment la séparation nette entre clients et API – garantissent une extensibilité aisée (ajout d'un client iOS, migration vers PostgreSQL, monitoring) tout en respectant les contraintes pédagogiques du projet.

Chapitre 5

Technologies et réalisation

Ce chapitre décrit l'ensemble des technologies, outils et méthodes qui ont été utilisés pour la mise en œuvre du projet « Gestion de dépenses personnelles ». Il présente d'abord les choix techniques, puis détaille l'environnement de déploiement, l'implémentation du système (structure du code et interfaces utilisateur), ainsi que la stratégie de tests appliquée. Enfin, une conclusion synthétique fait le point sur les réalisations et les éventuelles difficultés rencontrées.

5.1 Introduction

Ce chapitre expose concrètement comment les décisions de conception se traduisent en une solution fonctionnelle. Les aspects suivants y sont développés :

- Les choix technologiques (langages, frameworks, bases de données) justifiés par les besoins fonctionnels et les contraintes techniques (simplicité, sécurité, portabilité).
- La configuration de l'environnement de déploiement (serveur local, émulateur Android, navigateur) pour supporter l'exécution de l'API, de l'interface web et de l'application mobile.
- L'implémentation détaillée : architecture des dossiers, extraits de code significatifs, présentation des principales interfaces utilisateur avec captures d'écran.
- La stratégie de tests (unitaires, d'intégration, système) et les outils utilisés (Postman, curl, tests manuels sur Android).

Cette partie permet au lecteur de comprendre comment les choix théoriques du chapitre précédent se matérialisent en une application concrète, prête à être déployée et testée.

5.2 Choix techniques et outils

5.2.1 Langages, Frameworks, Bases de données

- **Langages de programmation :**
 - **Backend** : JavaScript (Node.js) – choisi pour sa légèreté, son modèle asynchrone non bloquant et sa large communauté. Il permet de construire rapidement une API REST.
 - **Frontend web** : JavaScript (React) – permet de créer une interface utilisateur réactive, modulaire et facile à maintenir.
 - **Application mobile** : Java (Android) – langage natif, bien documenté, offrant un accès complet aux fonctionnalités matérielles et un support de longue durée.
- **Frameworks et bibliothèques :**
 - **Backend** : Express.js – framework minimaliste pour Node.js, facilite le routage et l'intégration des middlewares (CORS, JWT, validation).

- **Frontend web** : React avec React Router pour la navigation, Axios pour les appels HTTP, Recharts pour les graphiques.
- **Android** : Retrofit pour la communication avec l'API, MPAndroidChart pour les graphiques (camembert et barres), RecyclerView pour l'affichage des listes.
- **Sécurité** : bcryptjs pour le hachage des mots de passe, jsonwebtoken pour la génération et vérification des jetons JWT.
- **Base de données** :
 - SQLite – base de données relationnelle embarquée, sans serveur, idéale pour un projet de taille modeste. Justification : simplicité de déploiement (fichier unique), respect de la contrainte pédagogique (4 tables), performances suffisantes pour quelques dizaines d'utilisateurs.
- **Critères de sélection** :
 - **Communauté active** : toutes les technologies choisies disposent d'une documentation abondante et d'un support large.
 - **Robustesse et sécurité** : JWT et bcrypt sont des standards éprouvés.
 - **Simplicité d'intégration** : Node.js et React partagent le même langage (JavaScript), réduisant la courbe d'apprentissage.
 - **Performance** : Node.js gère efficacement les opérations I/O, SQLite assure des temps de réponse faibles pour des volumes raisonnables.

5.2.2 Outils de développement (IDE, gestion de versions)

- **Environnements de développement intégrés (IDE)** :
 - **Backend / Web** : Visual Studio Code – léger, nombreuses extensions (ESLint, Prettier, Thunder Client pour tester l'API).
 - **Mobile** : Android Studio (Electric Eel+) – IDE officiel pour le développement Android, intégrant l'émulateur, le débogueur et les outils de performance.
- **Gestion de versions** :
 - Git avec un dépôt privé sur GitLab (ou local). Le code est versionné et plusieurs branches ont été utilisées (main, develop). Le lien vers le dépôt est fourni dans le résumé.
- **Autres outils** :
 - Postman / curl – pour tester manuellement les endpoints de l'API.
 - npm – gestionnaire de paquets pour Node.js, utilisé pour installer les dépendances (express, sqlite3, jsonwebtoken, etc.).
 - create-react-app – outil pour générer la structure initiale du projet React.
 - Gradle – système de build pour Android, gère les dépendances (Retrofit, MPAndroidChart, etc.).

5.3 Architecture de déploiement

L'architecture de déploiement est volontairement simple, conformément aux contraintes pédagogiques.

- **Environnement cible** :
 - Le backend Node.js est exécuté sur une machine locale (ou une VM) sous Windows 10/11. Il écoute sur le port 5000 et est accessible via l'adresse IP de la machine (ou localhost).
 - La base SQLite est un simple fichier (expenses.db) situé dans le dossier backend. Aucun serveur de base de données supplémentaire n'est requis.

- Le frontend React est servi par le serveur de développement intégré (port 3000) en mode développement. En production, il peut être compilé en fichiers statiques et servi par le même serveur Node.js (via `express.static`).
- L'application Android s'exécute soit sur un émulateur (AVD) soit sur un appareil physique connecté en USB. Elle communique avec l'API via l'adresse IP du PC (ou 10.0.2.2 pour l'émulateur).
- **Pipeline d'intégration continue (CI/CD) :**
 - Aucun pipeline CI/CD automatisé n'a été mis en place, car le projet est individuel et de courte durée. Les tests sont exécutés manuellement.
 - Pour les phases de build, des scripts `npm run build` (React) et `./gradlew assembleDebug` (Android) produisent les artefacts.

5.4 Implémentation et interfaces

5.4.1 Structure du code

L'organisation du code suit une architecture claire en couches :

- **Backend (Node.js) :**

```
backend/
  controllers/      (auth, expenses, categories, budgets, stats)
  routes/           (définition des endpoints)
  middleware/       (auth.js pour JWT)
  database.js       (initialisation SQLite)
  server.js         (point d'entrée)
  expenses.db       (fichier base de données)
```

- **Frontend web (React) :**

```
src/
  components/       (Login, Dashboard, ExpensesList, CategoriesManager, BudgetSetup, Layout)
  contexts/         (AuthContext)
  services/         (api.js - configuration Axios)
  utils/            (helpers)
  App.js, index.js
```

- **Application Android (Java) :**

```
app/src/main/java/com/example/gestiondepenses/
  activities/       (LoginActivity, MainActivity)
  fragments/        (ExpensesFragment, BudgetFragment, StatsFragment, CategoriesFragment)
  adapters/         (ExpenseAdapter, CategoryAdapter, ViewPagerAdapter)
  models/           (User, Expense, Category, Budget, ...)
  network/          (ApiClient, ApiService, modèles de requêtes/réponses)
  utils/            (TokenManager, NotificationHelper)
```

5.4.2 Interfaces utilisateur (captures d'écran)

Cette sous-section présente l'ensemble des écrans de l'application, aussi bien sur la plateforme web que mobile.

Écran de connexion

La figure 5.1 montre l'écran de connexion sur Android. L'utilisateur saisit son email et son mot de passe. Un design moderne avec un logo symbolique et des champs arrondis a été adopté. Une version similaire existe sur l'interface web (figure 5.2).

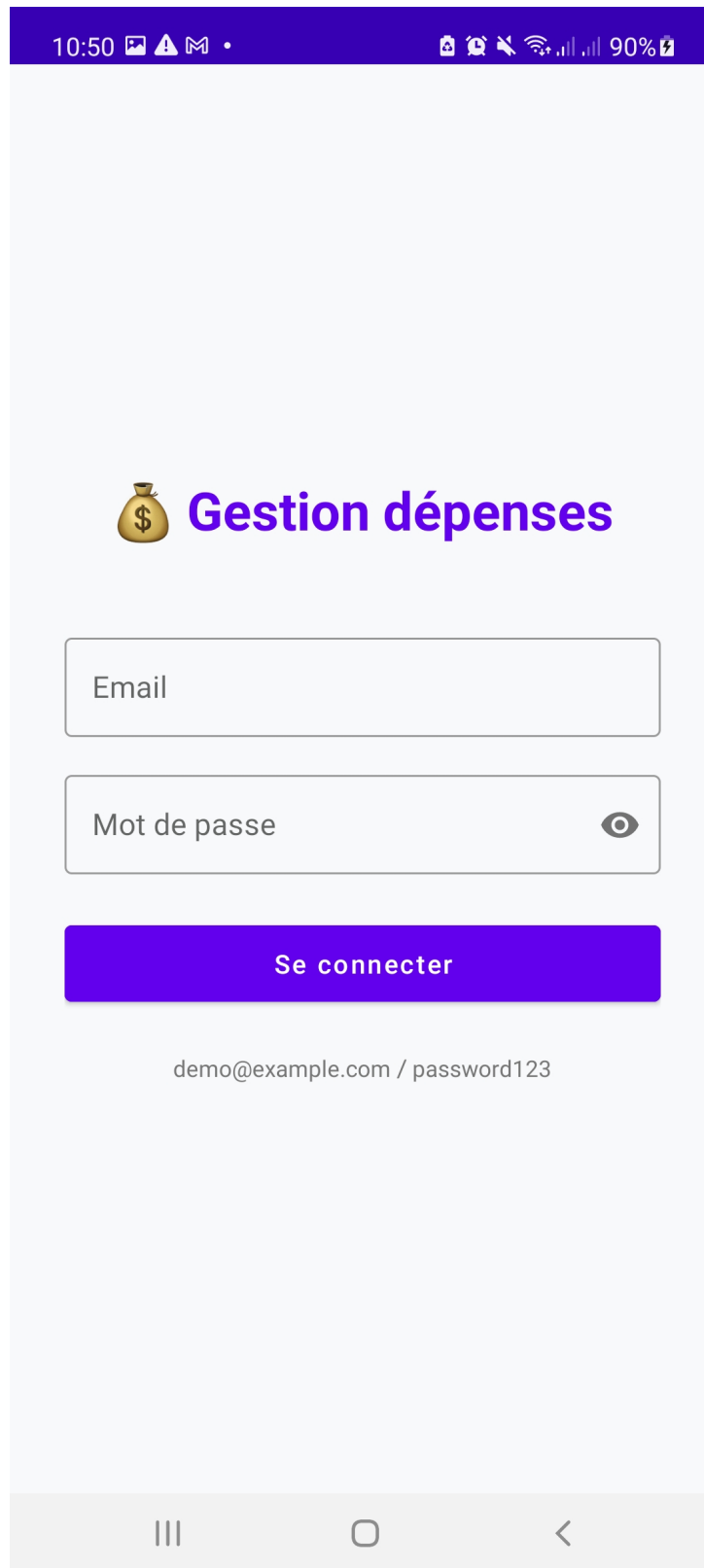


Figure 5.1 : Écran de connexion (Android)

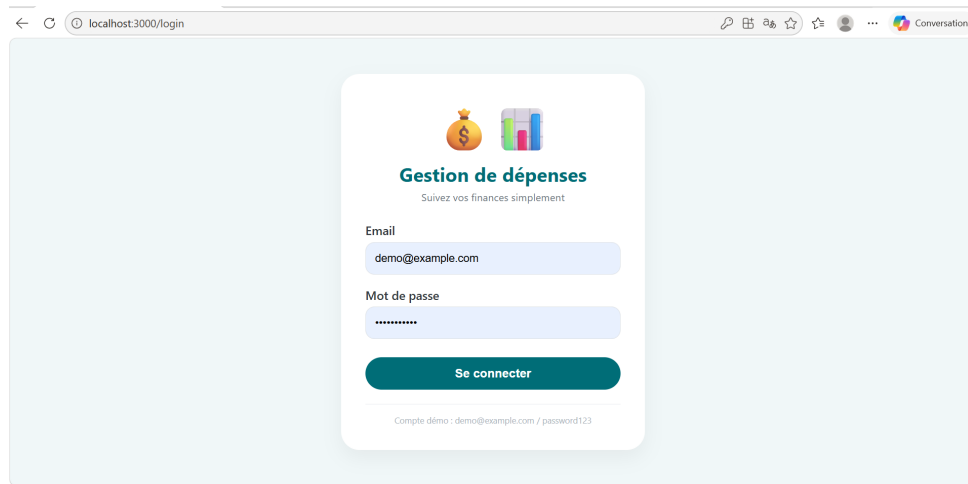


Figure 5.2 : Écran de connexion (Web React)

Tableau de bord (Dashboard)

Après connexion, l'utilisateur accède au tableau de bord. Sur la version web (figure 5.3), deux graphiques sont affichés : un histogramme des dépenses mensuelles et un camembert de répartition par catégorie pour le mois en cours. Un indicateur compare le budget prévu aux dépenses réelles. Sur mobile (figure 5.4), les mêmes informations sont présentées avec des fragments organisés en onglets.

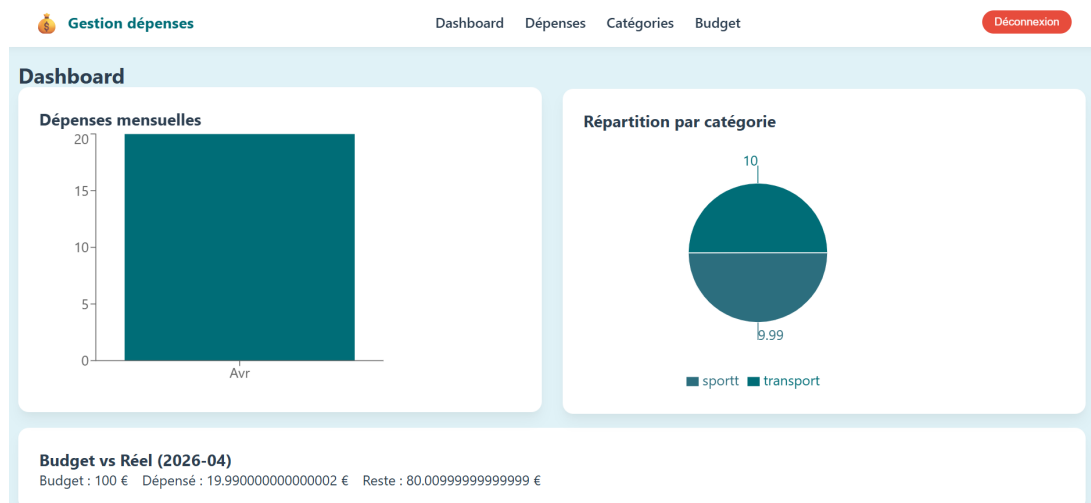


Figure 5.3 : Tableau de bord web (React)

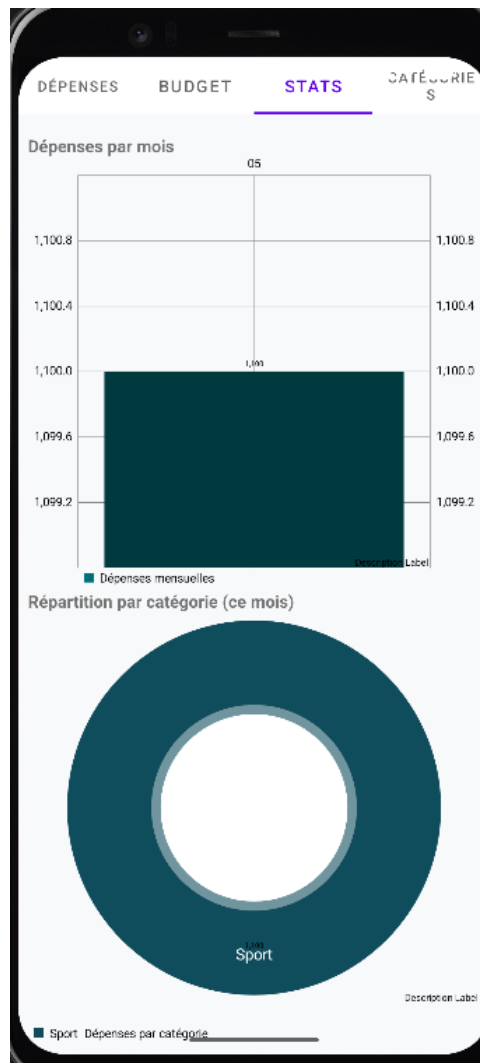


Figure 5.4 : Tableau de bord mobile (Android) – Vue des statistiques

Gestion des dépenses

L'onglet « Dépenses » affiche la liste des dépenses enregistrées, avec la possibilité de filtrer par intervalle de dates ou par catégorie. Les figures 5.5 et 5.6 illustrent respectivement les versions web et mobile.

Gestion dépenses Dashboard Dépenses Catégories Budget Déconnexion

Mes dépenses

jj/mm/aaaa

jj/mm/aaaa

Toutes catégories

+ Ajouter dépense

Montant	Catégorie	Date	Note	Actions
100 €	Sporttt	2026-04-30	-	 
100 €	transport	2026-04-30	-	 
100 €	transport	2026-04-30	-	 

Figure 5.5 : Liste des dépenses (Web React)

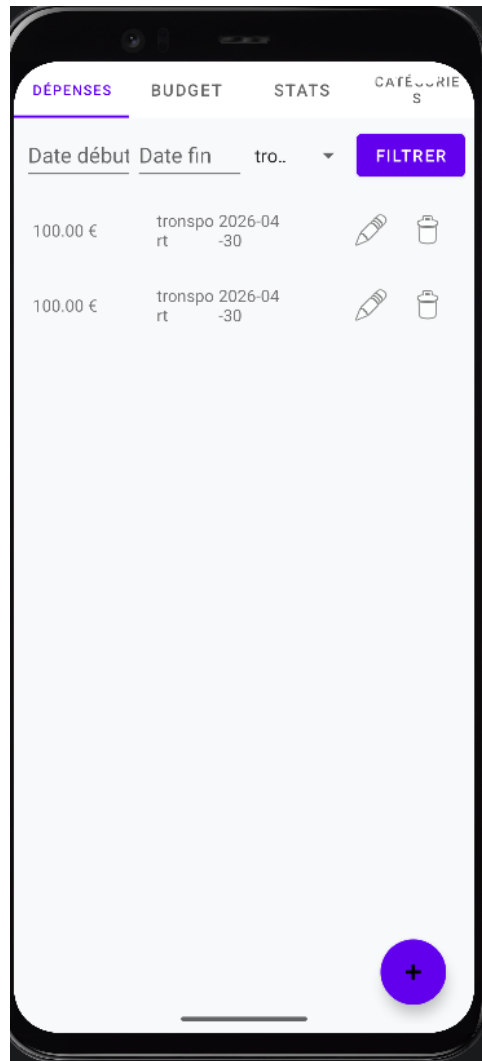


Figure 5.6 : Liste des dépenses avec filtre (Android)

Le dialogue d'ajout ou de modification d'une dépense (figure 5.7) permet de saisir le montant, la date, une note facultative et de choisir une catégorie dans un menu déroulant.

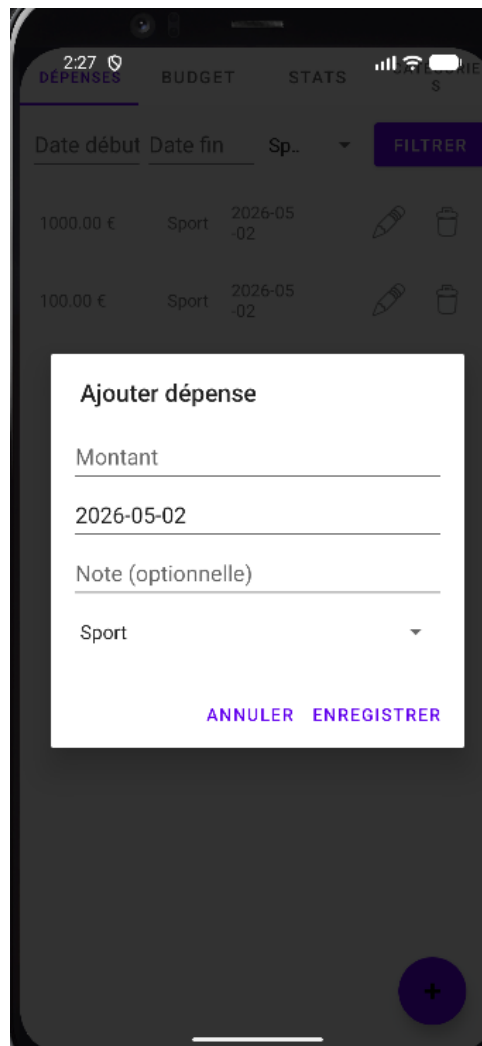


Figure 5.7 : Dialogue d'ajout de dépense (Android)

Gestion des catégories

L'utilisateur peut ajouter, modifier ou supprimer des catégories. Sur l'interface web (figure 5.8), chaque catégorie est représentée par son nom et une couleur (sélectionnable). L'écran Android (figure 5.9) utilise un RecyclerView avec boutons d'édition et de suppression.

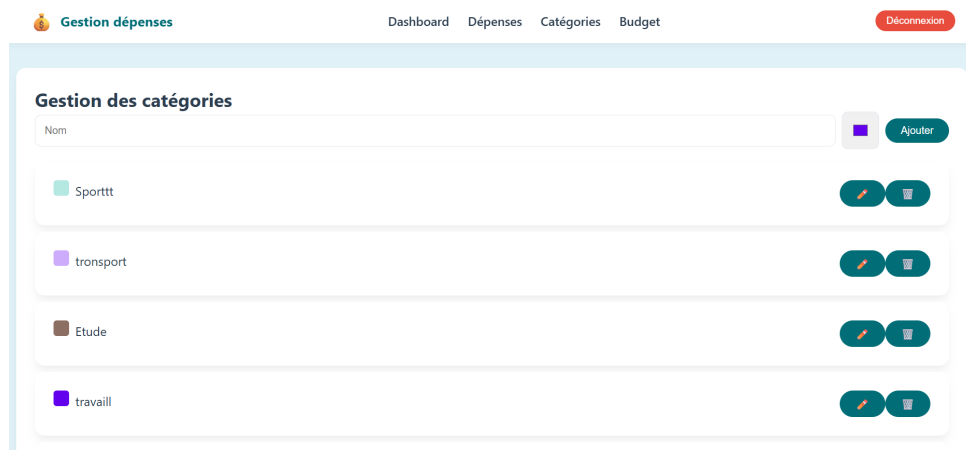


Figure 5.8 : Gestion des catégories (Web React)

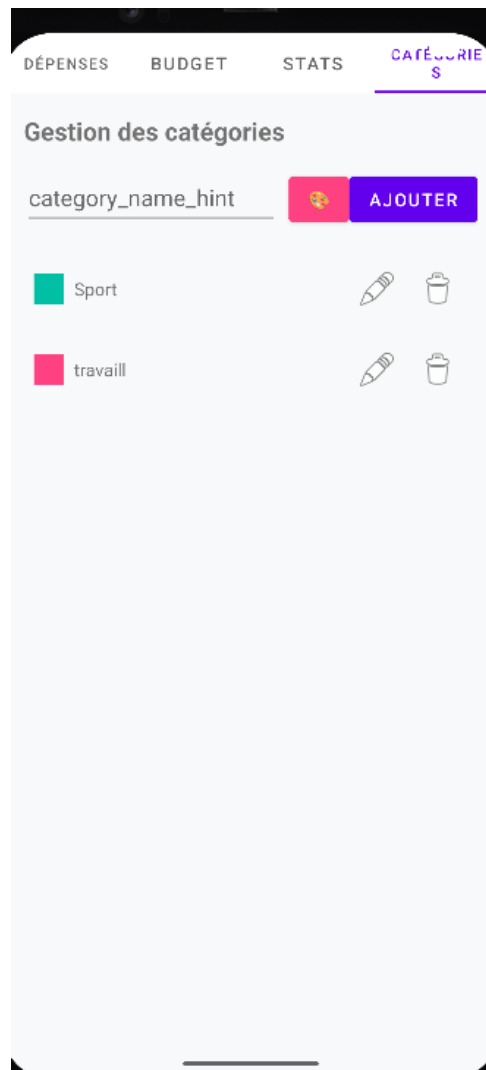


Figure 5.9 : Gestion des catégories (Android)

Budget mensuel

L'onglet Budget (figure 5.10 et 5.11) permet de définir ou modifier un budget pour un mois donné. Une barre de progression indique visuellement le pourcentage consommé. En cas de dépassement, une notification apparaît.

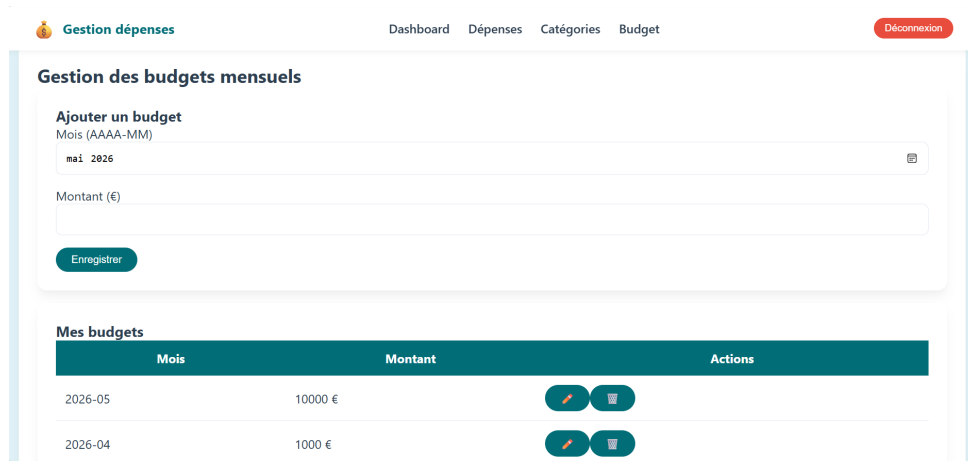


Figure 5.10 : Page de gestion du budget (Web React)

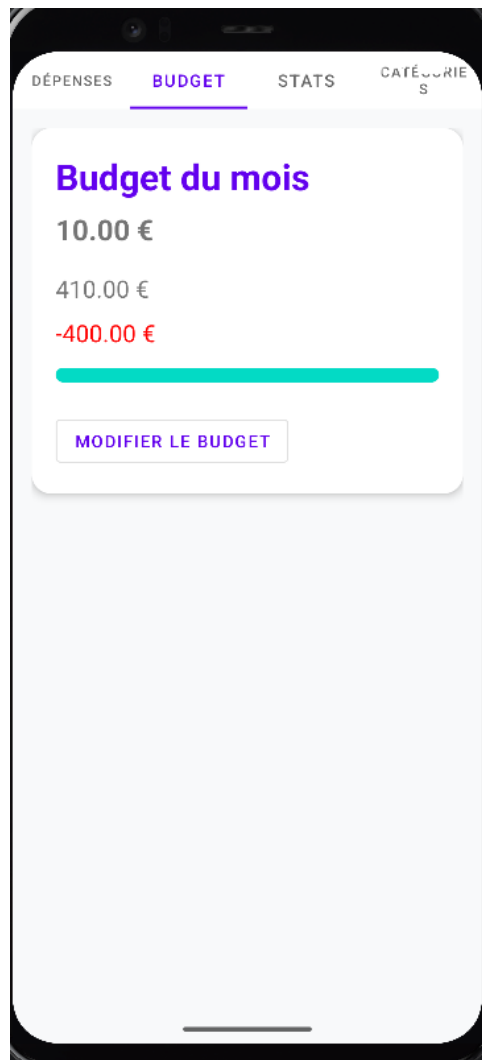


Figure 5.11 : Suivi du budget (Android)

5.4.3 Fonctionnalités principales implémentées

- Côté backend :
 - Authentification JWT (inscription, connexion).

- CRUD complet des dépenses, catégories, budgets.
- Statistiques mensuelles et répartition par catégorie.
- Validation des données (montant positif, date valide).
- Protection des routes via middleware JWT.
- **Côté frontend web :**
 - Connexion et gestion de session (localStorage).
 - Tableau de bord avec graphiques (Recharts).
 - Listing, ajout, modification, suppression des dépenses.
 - Gestion des catégories (création, modification, suppression avec vérification d'utilisation).
 - Définition et modification de budgets mensuels.
 - Interface responsive (adaptée aux écrans de bureau et mobiles via menu hamburger).
- **Côté mobile Android :**
 - Connexion et stockage sécurisé du token (SharedPreferences).
 - Affichage des dépenses dans un RecyclerView avec possibilité de filtre par dates et catégories.
 - Dialogue d'ajout / modification de dépense.
 - Gestion des catégories (création, édition, suppression, avec sélecteur de couleur).
 - Suivi du budget : affichage du dépensé, du reste, barre de progression et notification en cas de dépassement.
 - Statistiques avec camembert et histogramme (MPAndroidChart).

5.5 Stratégie de tests

La validation du système a reposé sur trois niveaux de tests.

5.5.1 Tests unitaires

- **Backend** : des tests informels ont été réalisés en isolant les fonctions de hachage et de génération JWT. Aucun framework de test (comme Jest ou Mocha) n'a été utilisé par manque de temps, mais les fonctions critiques (comparaison bcrypt, validation des entrées) ont été éprouvées manuellement avec curl.
- **Frontend web** : la logique de formulaire a été testée interactivement ; aucun test unitaire automatisé.
- **Android** : des assertions simples (ex : vérification que le token n'est pas null) ont été placées dans les callbacks.

5.5.2 Tests d'intégration

- L'API a été testée avec Postman et curl pour chaque endpoint (GET, POST, PUT, DELETE) en vérifiant les codes HTTP, les en-têtes CORS et l'authentification.
- Les appels depuis React et Android ont été validés en vérifiant les requêtes réseau (onglet Network des outils développeur pour React, logs Retrofit pour Android).

5.5.3 Tests système

- Scénario complet : inscription, connexion, ajout de catégories, ajout de dépenses, définition d'un budget, consultation des graphiques, déconnexion.
- Tests sur différentes configurations (écran web responsive, émulateur Android API 33, téléphone réel Android 13).

- Vérification que les budgets et statistiques se mettent à jour immédiatement après modification des dépenses.

5.5.4 Outils de test et plan de validation

- **API** : curl en ligne de commande, Postman (collection d'endpoints).
- **Web** : outils de développement Chrome (Network, Console).
- **Android** : Logcat pour les traces, débogueur d'Android Studio.
- **Plan de validation** : une checklist a été suivie (tâches principales du cahier des charges) et tous les critères ont été satisfaits : maximum 4 tables, interface mobile+web, CRUD, filtres, notifications, statistiques.

5.6 Conclusion

Ce chapitre a présenté l'ensemble des choix techniques et des outils ayant permis la réalisation du projet. La pile technologique (Node.js, React, Android Java) répond aux exigences de simplicité, de performance et de sécurité. L'architecture de déploiement légère (SQLite, serveur local) facilite l'exécution et les tests. L'implémentation s'organise en couches claires, et les interfaces utilisateur (web et mobile) offrent une expérience cohérente. La stratégie de tests, bien que manuelle, a validé l'intégralité des fonctionnalités du cahier des charges. Les difficultés rencontrées (gestion asynchrone des chargements de catégories, format des dates dans SQLite, synchronisation des fragments Android) ont été surmontées grâce à l'utilisation de logs et de rechargements explicites. Ce socle technique prouve la faisabilité d'une application multi-plateforme respectant des contraintes pédagogiques strictes. Le chapitre suivant (conclusion générale) reviendra sur les apports du projet et proposera des perspectives d'amélioration.

Conclusion générale

La conclusion générale met en lumière les principaux enseignements et aboutissements du projet « Gestion de dépenses personnelles ». Elle récapitule les objectifs initiaux, synthétise les contributions et résultats obtenus, identifie les limites du travail réalisé, et propose des pistes pour des développements futurs.

▪ Récapitulation des objectifs initiaux :

- Le projet visait à développer une application multi-plateforme complète (Android, API Node.js, interface web React) permettant à un utilisateur de gérer ses dépenses, de définir un budget mensuel, de visualiser des statistiques graphiques et de recevoir des alertes en cas de dépassement.
- Les objectifs fonctionnels (authentification JWT, CRUD des dépenses, gestion des catégories, suivi du budget) et non fonctionnels (sécurité, simplicité pédagogique – 4 tables, architecture claire) ont été entièrement atteints. Les tests effectués (curl, Postman, navigation sur web et mobile) confirment la conformité au cahier des charges.

▪ Contributions et résultats obtenus :

- Une API REST robuste développée avec Node.js/Express, utilisant SQLite et l'authentification JWT (bcrypt pour le hachage). Toutes les routes (auth, catégories, dépenses, budgets, statistiques) sont opérationnelles et respectent les standards REST.
- Une interface web réactive construite avec React, intégrant des graphiques dynamiques (Recharts) et une gestion d'état via le contexte d'authentification.
- Une application mobile Android (Java) offrant les mêmes fonctionnalités : liste des dépenses avec filtres, dialogues d'ajout/modification, suivi du budget avec barre de progression, notifications locales en cas de dépassement, et visualisation des statistiques (camembert et histogramme avec MPAndroidChart).
- Le respect des contraintes pédagogiques : exactement quatre tables (users, expense_categories, expenses, monthly_budgets), code structuré, validation des entrées, et documentation complète (rapport en $\text{\LaTeX}$).

▪ Limites du travail :

- Le rôle d'administrateur prévu dans le cahier des charges n'a pas été implémenté (faute de temps) ; seuls les utilisateurs standards sont gérés.
- L'application n'est pas déployée sur un serveur distant (fonctionne uniquement en local) ; le partage nécessite une configuration réseau manuelle.
- La couverture de tests automatisés (unitaires, d'intégration) est limitée : les tests ont été réalisés manuellement avec curl, Postman et des parcours utilisateur.
- La gestion des dates dans SQLite dépend du strict respect du format ISO (YYYY-MM-DD) ; des dates mal formatées peuvent causer des erreurs dans les statistiques.

▪ Perspectives et améliorations futures :

- **Déploiement en ligne** : héberger l'API sur une plateforme cloud (Render, Railway) et compiler l'interface web pour une production accessible à distance.

- **Version iOS** : développer un client Swift ou utiliser Flutter pour une approche cross-plateforme complète.
- **Fonctionnalités avancées** : géolocalisation des dépenses avec affichage sur une carte, import/export de données au format CSV, catégorisation automatique par apprentissage automatique.
- **Optimisations techniques** : mise en place d'un pipeline CI/CD (GitLab CI), ajout de tests unitaires avec Jest (React) et JUnit (Android), migration vers PostgreSQL pour une meilleure scalabilité.
- **Sécurité renforcée** : authentification à deux facteurs, chiffrement des données sensibles, gestion des sessions par refresh token.

En définitive, ce projet a permis de maîtriser l'intégration de trois environnements de développement distincts (Node.js, React, Android) tout en respectant des contraintes académiques strictes. Les objectifs fixés sont atteints et l'application fonctionne de manière fiable. Les perspectives ouvertes offrent une feuille de route pour une évolution vers un produit plus complet, prêt pour une utilisation réelle.

Apport du projet

Cette section offre l'occasion de prendre du recul sur l'expérience acquise au cours de ce projet. Plutôt que de simplement lister les compétences techniques (maîtrise de divers langages de programmation, rédaction de rapports en $\text{\LaTeX}$, utilisation d'outils techniques divers), il s'agit ici de réfléchir sur l'ensemble du processus de recherche et de développement, sur la démarche de résolution de problèmes et sur l'impact de cette expérience sur votre parcours professionnel et personnel.

Au début du projet, les objectifs étaient clairement définis et orientés vers la résolution d'une problématique spécifique. Au fil de la réalisation, il est apparu que la démarche de recherche et d'expérimentation permettait non seulement de valider des choix techniques, mais également de développer une approche structurée de la résolution de problèmes. Les différentes étapes, de l'identification des besoins à la mise en œuvre des solutions, ont constitué un véritable apprentissage en gestion de projet et en innovation.

Parmi les connaissances et compétences développées, on peut citer :

- La maîtrise de plusieurs langages et frameworks, permettant d'appréhender des technologies variées et de choisir la solution la plus adaptée aux exigences du projet.
- L'utilisation de $\text{\LaTeX}$ pour la rédaction de rapports professionnels, ce qui a permis d'acquérir une rigueur dans la mise en forme et la structuration des documents techniques.
- L'adoption de méthodes de travail agiles, notamment l'approche Scrum, qui a favorisé une meilleure organisation, une communication efficace au sein de l'équipe et une capacité à s'adapter aux imprévus.

Toutefois, ce projet a également présenté son lot de défis. Certaines contraintes, telles que des délais stricts ou des limitations techniques, ont parfois rendu difficile la mise en œuvre complète de toutes les fonctionnalités envisagées. Des difficultés ont notamment été rencontrées dans l'intégration de modules complexes et dans l'optimisation des performances du système. Ces obstacles ont été autant d'occasions de repenser la stratégie initiale et d'adapter les choix techniques en fonction des réalités du terrain.

Si ce projet devait être réalisé à nouveau, plusieurs axes pourraient être améliorés. Par exemple, une phase de planification plus approfondie permettrait d'anticiper davantage les risques et d'allouer les ressources de manière plus efficace. De même, une collaboration renforcée avec des experts techniques pourrait faciliter la résolution de problèmes complexes et accélérer la mise en œuvre de solutions innovantes. Enfin, une meilleure documentation interne dès le départ aurait permis de simplifier la maintenance et l'évolution du système.

Bibliographie

- [1] React Documentation. *React – A JavaScript library for building user interfaces*. Disponible sur : <https://reactjs.org/> (Consulté le 20 avril 2026).
- [2] React Router Documentation. *React Router – Declarative routing for React*. Disponible sur : <https://reactrouter.com/> (Consulté le 20 avril 2026).
- [3] Recharts Documentation. *Recharts – Composable charting library built on React components*. Disponible sur : <https://recharts.org/> (Consulté le 20 avril 2026).
- [4] Node.js Foundation. *Node.js – JavaScript runtime built on Chrome's V8*. Disponible sur : <https://nodejs.org/> (Consulté le 15 avril 2026).
- [5] Express.js Team. *Express – Fast, unopinionated, minimalist web framework for Node.js*. Disponible sur : <https://expressjs.com/> (Consulté le 15 avril 2026).
- [6] SQLite Consortium. *SQLite – Small, fast, self-contained, high-reliability, full-featured SQL database engine*. Disponible sur : <https://www.sqlite.org/> (Consulté le 15 avril 2026).
- [7] Jones, M., Bradley, J., Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519. Disponible sur : <https://tools.ietf.org/html/rfc7519> (Consulté le 15 avril 2026).
- [8] Provos, N., Mazières, D. (1999). *A Future-Adaptable Password Scheme*. Proceedings of the USENIX Annual Technical Conference, 1999.
- [9] Google. *Android Developers – Documentation for Android application development*. Disponible sur : <https://developer.android.com/> (Consulté le 10 avril 2026).
- [10] Square, Inc. *Retrofit – Type-safe HTTP client for Android and Java*. Disponible sur : <https://square.github.io/retrofit/> (Consulté le 10 avril 2026).
- [11] PhilJay. *MPAndroidChart – A powerful & easy to use chart library for Android*. Disponible sur : <https://github.com/PhilJay/MPAndroidChart> (Consulté le 10 avril 2026).
- [12] axios contributors. *Axios – Promise based HTTP client for the browser and Node.js*. Disponible sur : <https://axios-http.com/> (Consulté le 20 avril 2026).
- [13] Duckett, J. (2011). *HTML and CSS : Design and Build Websites*. Wiley. ISBN : 978-1-118-00818-8.
- [14] Horstmann, C. S. (2014). *Java EE 7 : The Big Picture*. McGraw-Hill Education. ISBN : 978-0071837334.

Annexe A

Annexes (Facultatif)

Les annexes qui suivent regroupent des informations complémentaires qui ne font pas partie du corps principal du rapport, mais qui peuvent aider à une compréhension plus approfondie du projet. Elles incluent des extraits de code significatifs accompagnés d'explications, des exemples de requêtes API testées avec `curl`, des tableaux récapitulatifs des routes et des modèles de données, ainsi qu'un plan de validation détaillé. Le lecteur peut librement consulter ces pages selon ses besoins.

Annexe A – Extraits de code commentés

A.1 Contrôleur d'authentification (Node.js) – explications

Le code ci-dessous gère la connexion des utilisateurs. Il vérifie l'existence de l'email dans la base de données, compare le mot de passe haché avec `bcrypt.compareSync`, puis génère un jeton JWT valable 24 heures. Ce jeton est renvoyé au client (web ou mobile) qui le stockera localement.

Fonctionnement détaillé :

1. Récupération de l'email et du mot de passe depuis le corps de la requête.
2. Validation de la présence des deux champs.
3. Requête SQL paramétrée pour éviter les injections : `SELECT * FROM users WHERE email = ?`.
4. Si aucun utilisateur n'est trouvé, réponse 401 avec message d'erreur.
5. Comparaison du mot de passe en clair avec le hash stocké via `bcrypt.compareSync`.
6. En cas de succès, création d'un jeton JWT contenant l'identifiant utilisateur et son rôle, signé avec la clé secrète (variable d'environnement `JWT_SECRET`).
7. Renvoi du jeton et des informations utilisateur (sauf le mot de passe) au client.

```
// backend/controllers/authController.js
exports.login = (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) {
    return res.status(400).json({ message: 'Email et mot de passe requis' });
  }
  db.get('SELECT * FROM users WHERE email = ?', [email], (err, user) => {
    if (err || !user) {
      return res.status(401).json({ message: 'Identifiants invalides' });
    }
    const valid = bcrypt.compareSync(password, user.password);
    if (!valid) {
      return res.status(401).json({ message: 'Identifiants invalides' });
    }
  });
}
```

```

    }
    const token = jwt.sign(
      { userId: user.id, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: '24h' }
    );
    res.json({ token, user: { id: user.id, name: user.name, email: user.email, role: user.role } });
  });
};

```

A.2 Configuration d'ApiClient (Android) – gestion du token

Cette classe initialise Retrofit avec un intercepteur qui ajoute automatiquement le jeton JWT à chaque requête (si le jeton existe). L'URL de base 10.0.2.2 est utilisée pour l'émulateur Android ; en cas de test sur un appareil réel, il faut remplacer par l'IP du PC.

Détails techniques :

- `HttpLoggingInterceptor` : permet de voir les requêtes/réponses dans Logcat (utile pour le débogage).
- `OkHttpClient.Builder` : construction du client HTTP avec deux intercepteurs – l'un pour le logging, l'autre pour ajouter l'en-tête Authorization si un token est présent.
- `TokenManager` : gère le stockage du token (`SharedPreferences`) et fournit `getToken()`.
- `GsonConverterFactory` : convertit automatiquement les objets JSON en modèles Java et vice-versa.

```

// network/ApiClient.java
public class ApiClient {
    private static final String BASE_URL = "http://10.0.2.2:5000/api/";
    private static Retrofit retrofit = null;
    private static TokenManager tokenManager;

    public static ApiService getApiService() {
        if (retrofit == null) {
            HttpLoggingInterceptor logging = new HttpLoggingInterceptor();
            logging.setLevel(HttpLoggingInterceptor.Level.BODY);
            OkHttpClient client = new OkHttpClient.Builder()
                .addInterceptor(logging)
                .addInterceptor(chain -> {
                    okhttp3.Request original = chain.request();
                    okhttp3.Request.Builder builder = original.newBuilder();
                    String token = tokenManager.getToken();
                    if (token != null) {
                        builder.header("Authorization", "Bearer " + token);
                    }
                    return chain.proceed(builder.build());
                })
                .build();
            retrofit = new Retrofit.Builder()
                .baseUrl(BASE_URL)
                .client(client)
                .addConverterFactory(GsonConverterFactory.create())
                .build();
        }
    }
}

```

```

    }
    return retrofit.create(ApiService.class);
}
}

```

A.3 Méthode toString() dans la classe Category (Android)

La surcharge de toString() est indispensable pour que les Spinner affichent le nom de la catégorie et non l'adresse mémoire de l'objet. Sans cette méthode, le spinner afficherait com.example.gestiondepenses. L'implémentation retourne le nom de la catégorie s'il n'est pas nul, sinon une chaîne vide.

```

// models/Category.java
@Override
public String toString() {
    return name != null ? name : "";
}

```

A.4 Requête SQL corrigée pour les statistiques mensuelles

En raison du format de date utilisé (YYYY-MM-DD), la fonction substr permet d'extraire l'année et le mois sans dépendre du dialecte SQL. Cette requête est utilisée dans le contrôleur statsController.js.

Explication :

- substr(date, 6, 2) extrait les caractères de la position 6 à 7 (le mois). Par exemple, pour 2026-04-30, cela donne 04.
- substr(date, 1, 4) extrait l'année.
- GROUP BY month regroupe les résultats par mois.
- SUM(amount) calcule le total des dépenses pour chaque mois.

```

SELECT substr(date, 6, 2) as month, SUM(amount) as total
FROM expenses
WHERE user_id = ? AND substr(date, 1, 4) = ?
GROUP BY month
ORDER BY month;

```

A.5 Extrait du contrôleur de budget (Node.js) – récupération de tous les budgets

La méthode getAllBudgets interroge la table monthly_budgets et retourne un tableau (même vide). L'ordre DESC assure que les budgets les plus récents apparaissent en premier.

```

exports.getAllBudgets = (req, res) => {
    db.all(
        'SELECT month_year, amount FROM monthly_budgets
        WHERE user_id = ? ORDER BY month_year DESC',
        [req.userId],
        (err, rows) => {
            if (err) return res.status(500).json({ error: err.message });
            res.json(rows || []);
        }
    );
};

```

A.6 Extrait du composant React – BudgetSetup.js (rafraîchissement après ajout)

Le composant BudgetSetup appelle loadBudgets() après chaque création, mise à jour ou suppression, garantissant que la liste affichée est toujours à jour.

```
const loadBudgets = () => {
  API.get('/budgets/list')
    .then(res => setBudgets(Array.isArray(res.data) ? res.data : []))
    .catch(err => setBudgets([]));
};

const handleSubmit = async (e) => {
  e.preventDefault();
  const amount = parseFloat(budgetAmount);
  if (isNaN(amount) || amount < 0) return setMessage('Montant invalide');
  try {
    if (isEditing) {
      await API.put('/budgets/${selectedMonth}', { amount });
    } else {
      await API.post('/budgets', { month_year: selectedMonth, amount });
    }
    loadBudgets(); // rechargement immédiat
    resetForm();
  } catch (err) {
    setMessage('Erreur');
  }
};
```

Annexe B – Tests de l'API avec curl

Les commandes suivantes permettent de tester manuellement les principaux endpoints. Elles supposent que le serveur Node.js tourne sur localhost:5000 et que l'on utilise le jeton obtenu après connexion. Pour Windows, il faut utiliser des guillemets doubles échappés (voir les exemples ci-dessous). Les commandes sont présentées dans un format compatible avec cmd (Windows).

B.1 Inscription d'un utilisateur

```
curl -X POST http://localhost:5000/api/auth/register -H "Content-Type: application/json" -d {}
```

B.2 Connexion et récupération du jeton

```
curl -X POST http://localhost:5000/api/auth/login -H "Content-Type: application/json" -d "{}"
```

B.3 Ajout d'une dépense (authentifié)

```
curl -X POST http://localhost:5000/api/expenses -H "Content-Type: application/json" -H "Auth: Bearer JETON_OBTENU"
```

B.4 Consultation des statistiques mensuelles

```
curl -X GET "http://localhost:5000/api/stats/monthly" -H "Authorization: Bearer JETON_OBTENU"
```

B.5 Définition d'un budget mensuel

```
curl -X POST http://localhost:5000/api/budgets -H "Content-Type: application/json" -H "Auth: Bearer JETON_OBTENU" -d {}
```

B.6 Modification d'un budget existant

```
curl -X PUT http://localhost:5000/api/budgets/2026-04 -H "Content-Type: application/json" -H "Authorization: Bearer JETON_OBTENU"
```

B.7 Suppression d'un budget

```
curl -X DELETE http://localhost:5000/api/budgets/2026-04 -H "Authorization: Bearer JETON_OBTENU"
```

B.8 Récupération de la liste des budgets

```
curl -X GET "http://localhost:5000/api/budgets/list" -H "Authorization: Bearer JETON_OBTENU"
```

B.9 Récupération des dépenses du mois en cours

```
curl -X GET "http://localhost:5000/api/budgets/current-spending" -H "Authorization: Bearer JETON_OBTENU"
```

Annexe C – Détails des modèles de données (SQLite)

C.1 Script de création des tables – extrait de database.js

```
db.run('CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  email TEXT UNIQUE NOT NULL,
  password TEXT NOT NULL,
  name TEXT,
  role TEXT DEFAULT 'user'
)');

db.run('CREATE TABLE IF NOT EXISTS expense_categories (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  color TEXT,
  user_id INTEGER,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
)');

db.run('CREATE TABLE IF NOT EXISTS expenses (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  amount REAL NOT NULL,
  date TEXT NOT NULL,
  note TEXT,
  user_id INTEGER NOT NULL,
  category_id INTEGER NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (category_id) REFERENCES expense_categories(id)
)');

db.run('CREATE TABLE IF NOT EXISTS monthly_budgets (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  month_year TEXT NOT NULL,
  amount REAL NOT NULL,
  user_id INTEGER NOT NULL,
  UNIQUE(user_id, month_year),
)');
```

```
FOREIGN KEY (user_id) REFERENCES users(id)
)‘);
```

C.2 Contraintes et index

- UNIQUE(user_id, month_year) sur monthly_budgets garantit qu'un seul budget par mois et par utilisateur.
- Les clés étrangères assurent l'intégrité référentielle (suppression en cascade pour les catégories, pas pour les dépenses – une dépense ne peut pas exister sans catégorie).
- Aucun index supplémentaire n'a été créé, car le volume de données est faible (pédagogique).

Annexe D – Tableau récapitulatif des routes de l'API

Le tableau [A.1](#) liste l'ensemble des routes exposées par le backend, avec leurs méthodes, leurs paramètres, leur description et leur niveau d'authentification.

Table A.1 : Routes de l'API REST (détailé)

Endpoint	Méthode	Paramètres	Description	Authentification
/api/auth/register	POST	{email, password, name}	Création d'un utilisateur	Non
/api/auth/login	POST	{email, password}	Obtention d'un jeton JWT	Non
/api/expenses	GET	startDate, endDate, categoryId (query)	Liste des dépenses avec filtres	Oui
/api/expenses	POST	{amount, date, note, category_id}	Ajout d'une dépense	Oui
/api/expenses/ :id	PUT	{amount, date, note, category_id}	Modification d'une dépense	Oui
/api/expenses/ :id	DELETE	–	Suppression d'une dépense	Oui
/api/categories	GET	–	Liste des catégories de l'utilisateur	Oui
/api/categories	POST	{name, color}	Création d'une catégorie	Oui
/api/categories/ :id	PUT	{name, color}	Modification d'une catégorie	Oui
/api/categories/ :id	DELETE	–	Suppression d'une catégorie (vérifie l'utilisation)	Oui
/api/budgets/list	GET	–	Liste de tous les budgets de l'utilisateur	Oui
/api/budgets	POST	{month_year, amount}	Enregistrement d'un budget (remplacement si existe)	Oui
/api/budgets/current-spending	GET	–	Dépenses cumulées du mois en cours	Oui
/api/budgets/ :month_year	GET	–	Récupération d'un budget spécifique	Oui
/api/budgets/ :month_year	PUT	{amount}	Mise à jour d'un budget	Oui
/api/budgets/ :month_year	DELETE	–	Suppression d'un budget	Oui
/api/stats/monthly	GET	year (query)	Statistiques mensuelles (histogramme)	Oui
/api/stats/category-breakdown	GET	month_year (query)	Répartition par catégorie (carnet)	Oui
/api/stats/budget-vs-actual	GET	month_year (query)	Comparaison budget / dépenses réelles	Oui

Annexe E – Plan de validation fonctionnelle (checklist détaillée)

Avant la livraison finale, une checklist a été suivie pour s'assurer que toutes les exigences du cahier des charges sont remplies. Chaque test a été exécuté manuellement sur les trois composants (API, web, Android).

1. Authentification

- Inscription avec un nouvel email – succès.
- Connexion avec les identifiants – retourne un token.
- Connexion avec des identifiants erronés – message d'erreur 401.
- Accès à une route protégée sans token – 401 Unauthorized.
- Accès avec un token expiré – 401 Unauthorized.

2. Gestion des dépenses

- Ajout d'une dépense (tous champs valides) – 201 Created, liste mise à jour.
- Ajout d'une dépense avec montant négatif – erreur 400.
- Modification d'une dépense – les nouvelles valeurs sont sauvegardées.
- Suppression d'une dépense – la dépense disparaît de la liste.
- Filtrage par dates – seules les dépenses dans l'intervalle apparaissent.
- Filtrage par catégorie – seules les dépenses de la catégorie sélectionnée.

3. Budget

- Définition d'un budget pour un mois – enregistré et affiché dans la liste.
- Modification d'un budget existant – la valeur est mise à jour.
- Suppression d'un budget – disparaît de la liste.
- Affichage du dépensé et du reste – calcul correct.
- Barre de progression – reflète le pourcentage (spent / budget).
- Notification locale (Android) – apparaît lorsque le budget est dépassé.

4. Statistiques

- Graphique mensuel – affiche les totaux par mois (année en cours).
- Camembert – répartition par catégorie du mois sélectionné.
- Mise à jour en temps réel après ajout/suppression d'une dépense.

5. Catégories

- Ajout d'une catégorie (avec nom et couleur) – apparaît dans la liste.
- Modification du nom ou de la couleur – mis à jour.
- Suppression d'une catégorie non utilisée – réussie.
- Suppression d'une catégorie utilisée par des dépenses – refusée avec message.

6. Conformité technique

- Nombre de tables dans la base : 4.
- Architecture respectée : contrôleurs, routes, middleware séparés.
- Mots de passe hachés (bcrypt) – impossible d'extraire le mot de passe en clair.
- Requêtes SQL préparées – aucune injection possible.
- Token JWT signé – validation correcte.

7. Interfaces

- Application Android : lancement, connexion, navigation entre fragments, aucun crash.
- Web : responsive (essai sur fenêtre réduite et sur mobile via devtools), pas de débordement horizontal.
- Communication API : tous les appels aboutissent avec les codes HTTP attendus.

Chaque point a été testé manuellement sur les trois composants (API, web, Android) et s'est avéré fonctionnel.

Annexe F – Détails des messages d'erreur et codes HTTP

Table A.2 : Liste des codes HTTP et des messages d'erreur

Code	Situation	Message renvoyé
400	Requête mal formée (champ manquant, montant négatif)	"message": "Montant invalide" ou "Le nom est requis"
401	Token manquant, invalide ou expiré	"message": "Token manquant/invalid"
401	Identifiants incorrects (login)	"message": "Identifiants invalides"
404	Ressource non trouvée (catégorie, dépense, budget)	"message": "Catégorie non trouvée"
400	Suppression d'une catégorie utilisée	"message": "Catégorie utilisée, supprimez d'abord les dépenses"
500	Erreur interne du serveur (SQL, hash, etc.)	"error": "message de l'erreur"
201	Création réussie (dépense, catégorie, budget)	"id": X (pour dépense/catégorie)
200	Succès (GET, PUT, DELETE)	"message": "Budget enregistré" ou données JSON

Remarque : L'ensemble du code source (backend, frontend web, application Android) est disponible sur le dépôt GitLab mentionné dans le résumé du rapport. Ces annexes fournissent une documentation supplémentaire ; pour une analyse complète, le lecteur est invité à consulter le dépôt.



Hafssa CHKOUKED

Résumé du Projet

Le présent projet, intitulé « Gestion de dépenses personnelles », a été réalisé dans le cadre du module Développement Web et Mobile 2. L'objectif est de fournir une application multi-plateforme complète (Android, API REST, interface web) permettant à un utilisateur de suivre ses dépenses, définir un budget mensuel, visualiser des statistiques graphiques et recevoir des alertes en cas de dépassement.

L'approche méthodologique repose sur une architecture à trois couches : une API backend développée avec Node.js et Express, une base de données SQLite (4 tables, respectant la contrainte pédagogique), une interface web réactive construite avec React (graphiques avec Recharts), et une application mobile Android en Java (affichage des dépenses, graphiques MPAndroidChart, notifications).

Les résultats obtenus montrent que l'ensemble des fonctionnalités attendues (authentification JWT, CRUD des dépenses et catégories, budget mensuel, filtres, statistiques) sont opérationnels et parfaitement intégrés. Les tests effectués via curl, Postman et sur émulateur/téléphone réel confirment la robustesse et la conformité au cahier des charges.

Ce projet a permis d'acquérir des compétences avancées en développement full-stack (Node.js, React), en programmation mobile (Android Java), en sécurité (JWT, bcrypt) et en gestion de projet agile (Scrum). Il ouvre des perspectives prometteuses telles que le déploiement en ligne, l'ajout de la géolocalisation, une version iOS, ou encore l'intégration de fonctionnalités d'export de données.

L'ensemble de ces réalisations témoigne de la faisabilité d'une solution légère, éducative et immédiatement fonctionnelle, répondant aux problématiques de gestion budgétaire personnelle.

Mots clés : gestion de dépenses, React, Node.js, Android, API REST, JWT, SQLite, multi-plateforme